

How to Make Any Computational Secret Sharing Scheme Adaptively Secure

George Lu
UT Austin
gclu@cs.utexas.edu

Brent Waters
UT Austin and NTT Research
bwaters@cs.utexas.edu

Abstract

Secret sharing (SS) is a foundational cryptographic primitive with diverse applications, including secure multiparty computation and conditional disclosure of secrets. While traditional schemes have primarily emphasized information-theoretic security, recent advancements have increasingly leveraged computational assumptions to achieve more efficient constructions and support broader access policies. Despite these successes, most existing computational secret sharing (CSS) schemes are limited to a static security model, where adversaries must commit to their choice of corrupted participants at the outset. A critical challenge in CSS lies in achieving adaptive security, where adversaries can dynamically select participants to corrupt, better reflecting real-world threat models.

In this paper, we present a novel transformation that converts any statically secure CSS scheme into an adaptively secure one while preserving the original access policy and computational assumptions, providing a framework for bridging the gap between static and adaptive security. Our construction introduces a multiplicative share size overhead of $O(n^2)$ where n is the number of parties. Additionally, we explore trade-offs in efficiency and security, offering more efficient adaptive CSS constructions for specific, restricted policy classes. This work addresses key limitations in the current landscape of CSS and paves the way for broader adoption of adaptively secure secret sharing in cryptographic applications.

1 Introduction

Secret sharing (SS) [Sha79, Bla79, ISN89] is a foundational cryptographic primitive which has been the focus of a great deal of research. It has widespread applications in secure multiparty computation (MPC), conditional disclosure of secrets (CDS), and beyond. At its core, secret sharing enables a dealer to distribute a secret among a group of participants such that only authorized subsets of participants can reconstruct the secret, while unauthorized subsets gain no information. This fundamental capability serves as a critical building block for numerous cryptographic protocols, underscoring the importance of designing efficient and secure SS schemes.

Classical secret sharing schemes predominantly focused on information-theoretic security (information-theoretic secret sharing - ITSS), where security holds unconditionally against computationally unbounded adversaries. Early works and many existing constructions achieve this stringent notion of security. Some more recent work has turned towards computational secret sharing (CSS), where security is based on computational hardness assumptions. This shift has allowed for schemes that efficiently capture a broader and more expressive classes of access policies not known in the information theoretic setting [Yao89, VNS⁺03], as well as enabling secret sharing with a variety of additional properties such as verifiable secret sharing (VSS) [Fel87, Ped91], homomorphic secret sharing (HSS) [BGI16, BGI⁺18], robust secret sharing [Kra93, RB07], succinct secret sharing [ABI⁺23], and more. By relying on computational assumptions, CSS can overcome limitations inherent in ITSS, such as share sizes inherently growing with message space [Kra93].

A critical consideration in CSS is the distinction between static and adaptive security. In the secret sharing security game, an adversary specifies an access policy and a pair of secrets s_0 and s_1 . In addition, they specify a set of

parties which do not satisfy the access policy, and receive the shares of those parties. The critical difference is that in the static security game, the adversary must commit to the set of corrupted participants at the start of the security game, while adaptive security allows the adversary to make corruption queries one at a time, dynamically deciding which party to next corrupt based on already corrupted shares. The adaptive setting reflects a more realistic and flexible adversarial model, accommodating attackers with resource constraints or evolving strategies. For example, an attacker might have resources that can let him corrupt or hack a limited number of parties and will employ an adaptive strategy to dynamically determine which parties to corrupt. Adaptive security is thus widely regarded as the "right" definition, as it captures the broadest class of attackers and aligns with real-world threat scenarios. However, achieving adaptive security in CSS often presents significant technical challenges, as evidenced by the predominant focus on static security in the literature.

The difficulty of achieving adaptively secure CSS can be illustrated by examining existing constructions. For example, we can look at the original CSS constructions for monotone boolean circuits by Yao [Yao89] and Vaikuntanathan et al. [VNS⁺03]. At a high level, this construction is reminiscent of a stripped down version of Yao's garbled circuits [Yao82, Yao86], where a secret key value is encoded on every wire of an access policy circuit. Beginning with the shared secret, which is encoded in the output wire of the policy circuit, the values are propagated down the circuit through the gates. Although AND and OR gates can be secret shared information-theoretically, as evidenced by the ability for ITSS to support monotone boolean formula [BL88], Yao [Yao89] and Vaikuntanathan et al. [VNS⁺03] leverage computational assumptions in order to enable gates with multiple FAN-OUT. This is modeled by an explicit one input, two output 'FAN-OUT' gate, which is implemented by simply encrypting the two output wires under a randomly sampled secret key, which will be used as the input wire value. To reason about security, the reduction relies on the fact that at the input-level of the circuit, wires which are not satisfied in the monotone circuit are information-theoretically hidden. As such, any input wires to FAN-OUT gates which are not satisfied are not given out. This allows us to computationally switch the outputs of such FAN-OUT gates to encryptions of the zero string, at which point any wires dependent on these values are hidden. However, notice that this proof strategy critically relies on knowing which wires are left unsatisfied in the security game. But when attempting to show adaptive security, the reduction needs to commit to the FAN-OUT encryptions before knowing this information, making it unclear how to proceed with this reduction.

A direct way to bridge this gap may be to leverage generic techniques which give adaptive security, such as complexity leveraging [JKK⁺17]. While this can work, it generally comes with undesirable downsides, such as introducing new assumptions which deviate from that of the underlying static scheme - requiring subexponential security in the case of complexity leveraging. In the case of certain constructions, such as the CSS for monotone circuits of [Yao89, VNS⁺03], utilizing random oracles could also allow for adaptive security, but this again introduces a new heuristic. The situation is similar when looking at more domain-specific techniques for achieving adaptivity, such as dual-system techniques from attribute-based encryption (ABE) [LOS⁺10], which, despite having a superficial similarity to SS in both primitives controlling access to secret information based on an access policy, lacks the decomposability of secret sharing and requires public key assumptions. In addition, many of these techniques are reliant on specific constructions of the underlying primitive, making a generic solution seem unlikely.

Our Results In this work, we present a novel approach to constructing adaptively secure CSS schemes. Specifically, we show a transformation that converts any statically secure CSS into an adaptively secure one for the same access policy and under the assumption of the polynomial hardness of the original scheme plus one way functions. Our construction introduces a hierarchical layering technique, where each of the n users receives shares across n layers. The key idea is that users initially are only given access to their first layer of shares in the clear, with their shares for subsequent layers being encoded through secret sharing using the prior layer. This structure enables us to prove adaptive security by incrementally stripping away the shares of higher 'layers' for later corruption queries the adversary makes. This gives the *first* adaptively secure CSS from polynomial hardness assumptions for several policy families, including monotone circuits [Yao89, VNS⁺03] and monotone NP [KNY14].

Our transformation achieves adaptive security with a share size $n^2 \times$ that of the underlying static scheme. We further discuss trade-offs and optimizations, including reducing overhead for bounded collusion settings. A detailed technical overview and extensions are presented in Section 1.1 and Section 1.2, respectively.

1.1 Technical Overview

Our construction hinges on two main insights. First, we can consider a notion of semi-adaptive security where an attacker is allowed to adaptively corrupt individual user shares and then subsequently see the public share (global information available to all parties). Once the public share is viewed, no more individual shares can be corrupted. A very basic transformation allows us to go from static to this form of semi-adaptive security. Here, user shares consist of private encryption keys. The public share of the new scheme consists of the original public share plus the individual shares of the original scheme encrypted under the corresponding secret keys.

Our second transformation upgrades semi-adaptive security to fully adaptive by recursively secret sharing the shares of a semi-adaptive scheme within another instance of the semi-adaptive secret sharing scheme. Now, instead of directly receiving a secret sharing of the message, each user obtains a share which enables them to recover another share (when participating with a satisfying set of other users), which in turn allows them to recover another secret share, and so on. By repeating this n times (where n is the number of users), before finally encoding a share of the message, we can argue adaptive security of our ‘layered’ scheme via a hybrids which incrementally decreases the number of ‘layers’ containing meaningful information.

Semi-Adaptively Secure CSS Our first transformation will realize a semi-adaptively secure CSS scheme from a statically secure one. In addition, the semi-adaptive scheme resulting from the transformation will satisfy a form of decomposability where we can separate the generation of user shares and public information such that user shares are *independent* of the policy and secret. Syntactically, we capture this requirement as splitting our Share algorithm into two components. First, the ShareKey algorithm, which only takes in the security parameter and number of parties supported by the access policy n to output the user shares $(sh_1, \dots, sh_n)$ as well as a secret key sk . The next algorithm ShareMsg, will take in the secret key sk , the access policy P and secret to be shared s to output the public information ps . We will require this decomposability to efficiently realize our following semi-adaptive to adaptive transformation. We note that many existing computational secret sharing schemes are naturally semi-adaptive and decomposable, but for generality, our transformation show how these can be achieved generically with only symmetric key encryption (SKE).

- $\text{ShareKey}(1^\lambda, 1^n)$: Our first share algorithm generates n SKE secret keys $\{sk_i\}_{i \in [n]}$ and outputs user shares $sh_i = sk_i$.
- $\text{ShareMsg}(sk, P, s)$: To generate the public information, we first compute the statically secure shares and public information as $S.ps, \{S.sh_i\}_{i \in [n]}$ and simply output our public information as

$$ps = (S.ps, \{\text{Encrypt}(sk_i, S.sh_i)\}_{i \in [n]}).$$

Reconstruction proceeds by simply decrypting the appropriate shares and running the reconstruction of the underlying static CSS scheme. The semi-adaptive security of this construction follows its structure. Since the static CSS scheme is only present in the public information of our new scheme, by the time it is revealed in the semi-adaptive security game, we know exactly which shares are not going to be corrupted, which are exactly the ones we did not give out secret keys to. As a result, we can computationally replace all uncorrupted shares of the underlying scheme with encryptions of the 0 string, relying on the security of our symmetric key encryption. After these are all replaced, security follows from static security of the underlying CSS scheme.

Semi-Adaptive to Adaptive CSS Once we have a semi-adaptive and decomposable CSS scheme, we can turn this into an adaptively secure scheme through the following transformation.

- $\text{Share}(1^\lambda, P, s)$: Our Share algorithm instantiates $n + 1$ ‘levels’ of the semi-adaptive decomposable scheme (with secret keys $(sk^0, \dots, sk^n)$) under the same policy P where user shares for the j^{th} level for $j \in [0, n]$ are $\{sh_i^j\}_{i \in [n]}$. However, rather than giving each user i all $n + 1$ of their shares in the clear, the adaptive user share for user i consists of sh_i^n , their share of the last level in the clear, as well as ps_i^{j-1} for $j \in [n]$, a secret sharing

¹Note that although we refer to this as ps , it is only the public share of the underlying semi-adaptive scheme, and is part of the user share of in the compiled adaptive scheme.

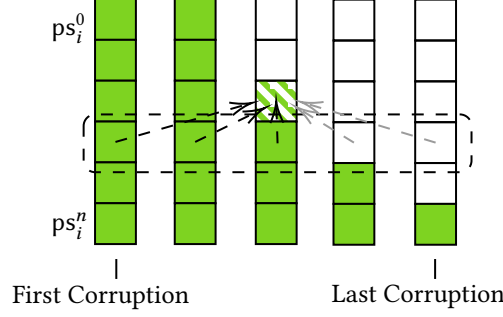


Figure 1: Structure of shares within our security proof. Green squares indicate ones where share ps_i^{j-1} properly encode sh_i^j , while white squares have been replaced with encodings of the zero string. Notice that all the shares encoding information dependent on our challenge share (dashed) in subsequent corruption have been replaced with encodings of the zero string, allowing us to use semi-adaptive security.

of sh_i^{j-1} under the j^{th} level, computed as $ps_i^j \leftarrow \text{ShareMsg}(sk^j, P, sh_i^{j-1})$. The secret s is only shared in the 0^{th} level as $T \leftarrow \text{ShareMsg}(sk^0, P, s)$, which also acts as the public share of our adaptive scheme.

- $\text{Recon}(P, x, ps, \{sh_i\}_{i \in S_x})$: To recover the message, we can simply recursively use the shares of the j^{th} semi-adaptive instance to decode ps_i^j , recovering the shares of the $(j-1)^{th}$ instance. Once we obtain the shares of the last instance, we can directly use this to recover the message from T .

To argue security of our construction, we leverage the fact that semi-adaptive security allows us to remove information from the *last* share corrupted by the adversary. Suppose user i is the last one to be corrupted, then this user share consists of a semi-adaptive user share sh_i^n and semi-adaptive public shares $ps_i^1, \dots, ps_i^n$, we can replace all ps_i^j from an encoding of sh_i^{j-1} to an encoding of the 0 string. But now, we can look at the *second to last* corruption query made by the adversary on some other party i' . Although we cannot directly alter $ps_{i'}^n$ using semi-adaptive security since $sh_{i'}^n$ will need to be given out in the following corruption query, we *can* set $ps_{i'}^1, \dots, ps_{i'}^{n-1}$ to encodings of the 0 string, since for $j \in [n-1]$, the only components dependent on $sh_{i'}^j$ was $ps_{i'}^{j-1}$, which was already replaced with an encoding of the 0 string. This means for $j \in [n-1]$, the second to last corruption query effectively functions as the final corruption query, allowing us to replace the secret using semi-adaptive security [Section 1.1](#).

Ultimately in our hybrids, the challenger will encode at a degrading ‘staircase’ of share information, where the first corruption query will honestly generate the semi-adaptive shares of the outermost n semi-adaptive levels, the second corruption will honestly generate semi-adaptive shares of the outermost $n-1$ levels, and so on, erasing the shares encoded in every other instance. At this point however, we can observe that we have *completely* removed any user shares of the innermost 0^{th} level, which was only component which encodes the secret s . Security now follows from semi-adaptive (and indeed static) security of the 0^{th} level.

1.2 Extensions

In addition to our generic construction, we also present some extensions which can provide adaptive computational secret sharing schemes with lower overhead for more specialized settings.

Trading Size for Reduction Tightness Our construction admits a generic trade-off between reduction tightness and parameter size. Much like the work of [\[DN98\]](#), this is achieved by allowing our reduction to do a bounded amount of ‘guessing’ at each level of the scheme, allowing us to reduce the total layers of our scheme. Concretely, we can instantiate a scheme with (n/c) levels (rather than n) at the cost of a n^{-c+1} loss in reduction tightness relative to the underlying semi-adaptive scheme.

Bounded-Collusion Adversaries Oftentimes, secret sharing is considered in a setting where adversaries have the ability to only corrupt a limited number of parties, either as part of a larger protocol where such limitations are inherent or where the secret sharing is instantiated with policies that are always satisfied with sufficiently many parties. In these situations with collusion bound q , we can simply reduce the number of ‘layers’ of our adaptive construction to $q + 1$. The security argument proceeds almost identically to that of the fully adaptive transformation, where we similarly erase more and more share information in later corruption queries. For a version of our main construction parameterized by reduction tightness and collusion bound, see [Appendix A](#).

Threshold Access Structures We observe that for threshold access structures, we can execute an even simpler delayed binding argument to transform any CSS scheme into an adaptively secure one with minimal overhead. Exploiting the ‘symmetry’ of threshold access structures, we can randomly permute the shares of the underlying scheme. This allows the reduction to simply request a random subset of shares from the static challenger, which it can dynamically bind to adversary’s corruption queries the using the permutation to ‘explain’ their choices. While perfectly (and thus adaptively) secure secret sharing has long been known for threshold access structures [Sha79], computational assumptions have been applied even in this setting to attain schemes with additional properties. Our generic transformation [Section 5.1](#) gives a way to retain some of these properties while upgrading the security.

Succinct Secret Sharing A central topic in the study of secret sharing has been the relation between share size and policy size. In particular, constructing secret sharing schemes with share size smaller than natural ‘representations’ of the access policy has proven to be a difficult problem, both information-theoretically and computationally. In the computational setting, the recent work of [ABI⁺23] been able to instantiate secret sharing schemes with share sizes *polynomially* smaller than the representation size for specific families of access policies. Like many other works in the area, their constructions are shown statically secure, but unfortunately, most of their constructions result in a total share size $O(n)$ smaller than the access policy, meaning our generic transformation’s $O(n^2)$ overhead, when applied to such schemes, loses any meaningful notion of succinctness. In [Section 5.2](#), we demonstrate how the [ABI⁺23] construction of CSS for graph access policies can be shown to be adaptively secure, giving a simultaneously succinct and adaptively secure secret sharing scheme.

Other Properties In addition to more expressive policy classes, computational assumptions are also utilized in the context of secret sharing to achieve additional properties such as verifiability [Fel87, Ped91] or homomorphism [BGI⁺18]. Our transformations do not necessarily preserve such properties, but believe it can be a useful pathway to achieving CSS schemes which are simultaneously adaptively secure and enjoy some of these additional computational properties through either additional generic techniques or property/construction specific analysis.

2 Preliminaries

Notation We use $[n]$ to refer to the set $\{1, 2, \dots, n\}$. We say a function $\text{negl}(\cdot)$ is negligible if for any polynomial $\text{poly}(\cdot)$, $\text{negl}(\lambda) < \frac{1}{\text{poly}(\lambda)}$ for sufficiently large $\lambda \in \mathbb{N}$.

2.1 Symmetric-Key Encryption

Definition 2.1 (Symmetric Key Encryption). Let λ be a security parameter. A symmetric key encryption scheme supporting message space $\mathcal{M} = \{\mathcal{M}_\lambda\}$ is a tuple of efficient randomized algorithms $\Pi_{\text{SKE}} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt})$:

- $\text{KeyGen}(1^\lambda) \rightarrow \text{sk}$: On input security parameter λ , the key generation algorithm samples a secret key sk .
- $\text{Encrypt}(\text{sk}, \mu) \rightarrow \text{ct}$: On input secret key sk and message μ , the encrypt algorithm outputs a ciphertext ct .
- $\text{Decrypt}(\text{sk}, \text{ct}) \rightarrow \mu \cup \perp$: On input secret key sk and ciphertext ct , the decrypt algorithm recovers the message μ (or $\perp$ representing a decryption error).

We require the following properties:

- **Correctness:** For all security parameters λ , messages $\mu \in \mathcal{M}$ and sk in the support of KeyGen , we have that

$$\Pr[\text{Decrypt}(\text{sk}, \text{ct}) = \mu : \text{ct} \leftarrow \text{Encrypt}(\text{sk}, \mu)] = 1.$$

- **Indistinguishability under Chosen Plaintext Attack (IND-CPA) Security:** For a bit $b \in \{0, 1\}$ and an adversary $\mathcal{A}$, define the symmetric-key security game $\text{ExptSKE}_{\mathcal{A}}(\lambda, b)$ as follows:

1. The challenger samples secret key $\text{sk} \leftarrow \text{KeyGen}(1^\lambda)$
2. Algorithm $\mathcal{A}(1^\lambda)$ can make as many encryption queries of the following form as it likes: $\mathcal{A}$ submits a message m , and the challenge responds with $\text{ct} \leftarrow \text{Encrypt}(\text{sk}, m)$.
3. Adversary $\mathcal{A}$ submits a challenge query in the form of two messages m_0, m_1 . The challenger replies with $\text{ct}^* \leftarrow \text{Encrypt}(\text{sk}, m_b)$
4. Adversary $\mathcal{A}$ can continue to make encryption queries as in Step 3.
5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$, which is the output of the experiment.

We require that for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{ExptSKE}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{ExptSKE}_{\mathcal{A}}(\lambda, 1) = 1]| = \text{negl}(\lambda).$$

2.2 Computational Secret Sharing

Definition 2.2 (Access Structure [Bei96]). For an $n \in \mathbb{N}$ let $[n]$ denote the set of integers from 1 to n , let $2^{[n]}$ denote the power set of $[n]$ (the set of all subsets of $[n]$). An access structure on $[n]$ is a set $\mathbb{A} \subseteq 2^{[n]} \setminus \emptyset$ of non-empty subsets of $[n]$. We refer to the elements of $\mathbb{A}$ as the *authorized* sets and those not in $\mathbb{A}$ as the *unauthorized* sets. We say an access structure is *monotone* if for all sets $S_1, S_2 \in 2^{[n]}$, if $S_1 \in \mathbb{A}$ and $S_1 \subseteq S_2$, then $S_2 \in \mathbb{A}$. We say a program P implements an access structure if P takes as input a set $X \in 2^{[n]}$ and $P(X) = 1$ iff $X \in \mathbb{A}$ (and $P(X) = 0$ otherwise).

We now define the main notion of computational secret sharing. While (computational) secret sharing has been defined in many prior works, our most closely follows that of [ABT⁺23].

Definition 2.3 (Computational Secret Sharing). Let λ be a security parameter. A computational secret sharing (CSS) scheme with supporting programs $\mathcal{P} = \{P_\lambda\}$, secret space $\mathcal{S} = \{\mathcal{S}_\lambda\}$ is a tuple of efficient algorithms $\Pi_{\text{CSS}} = (\text{Share}, \text{Recon})$:

- $\text{Share}(1^\lambda, P, s) \rightarrow (\text{ps}, \text{sh}_1, \dots, \text{sh}_n)$: On input security parameter λ , program $P \in \mathcal{P}$, and secret $s \in \mathcal{S}$, the share algorithm generates a set of shares $(\text{ps}, \text{sh}_1, \dots, \text{sh}_n)$ where ps is the public information, and $\text{sh}_1, \dots, \text{sh}_n$ are the shares distributed to the n parties described in program P .
- $\text{Recon}(P, X, \text{ps}, \{\text{sh}_i\}_{i \in X}) \rightarrow s \cup \perp$: On input program P , input x , public information ps , and set of user shares $\{\text{sh}_i\}$ on indices $i \in X$, the reconstruction algorithm outputs the secret s (or $\perp$ for a reconstruction error).

We refer to share size $\text{shsize} = \text{shsize}(\lambda, |P|)$ to denote the maximum size of individual shares $\max\{|\text{sh}_i| \mid i \in [n]\}$ in the above scheme and public information size to denote the size of ps . We require the following properties:

- **Correctness:** For all security parameters λ , secrets $s \in \mathcal{S}$ and programs $P \in \mathcal{P}$ and inputs x where $P(x) = 1$, we have that

$$\Pr[\text{Recon}(P, X, \text{ps}, \{\text{sh}_i\}_{i \in X}) = s : \text{Share}(1^\lambda, P, s) \rightarrow (\text{ps}, \text{sh}_1, \dots, \text{sh}_n)] = 1$$

- **Adaptive Security:** For a bit $b \in \{0, 1\}$ and an adversary $\mathcal{A}$, define the secret sharing security game $\text{ExptAdaptiveSS}_{\mathcal{A}}(\lambda, b)$ as follows:

1. Algorithm $\mathcal{A}(1^\lambda)$ chooses a program P on n inputs and a pair of secrets s_0, s_1 to send to the challenger. The challenger samples $(\text{ps}, \text{sh}_1, \dots, \text{sh}_n) \leftarrow \text{Share}(1^\lambda, P, s_b)$ and responds with ps .

2. For as many times as desired, the adversary $\mathcal{A}$ sends an index $i \in [n]$, and the challenger responds with sh_i .
3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $P(X) = 0$, the experiment outputs b' . Otherwise, the experiment outputs 0.

We say Π_{CSS} is adaptively secure if, for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{ExptAdaptiveSS}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{ExptAdaptiveSS}_{\mathcal{A}}(\lambda, 1) = 1]| = \text{negl}(\lambda).$$

We also define a weaker, static notion of security which is what is typically considered in prior works on CSS. This is identical to our adaptive definition, except the adversary submits *all* shares it intends to corrupt at the beginning of the security game.

- **Static Security:** For a bit $b \in \{0, 1\}$ and an adversary $\mathcal{A}$, define the secret sharing security game $\text{ExptStaticSS}_{\mathcal{A}}(\lambda, b)$ as follows:

1. Algorithm $\mathcal{A}(1^\lambda)$ chooses a policy P , an unauthorized set X where $P(X) = 0$ and a pair of secrets s_0, s_1 .
2. The challenger samples $(\text{ps}, \text{sh}_1, \dots, \text{sh}_n) \leftarrow \text{Share}(1^\lambda, P, s_b)$ and returns $\text{ps}, \{\text{sh}_i\}_{i \in S_X}$.
3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$, which is the output of the experiment.

Like before, we say Π_{CSS} is statically secure if, for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{ExptStaticSS}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{ExptStaticSS}_{\mathcal{A}}(\lambda, 1) = 1]| = \text{negl}(\lambda).$$

3 Decomposable Secret Sharing

To construct our adaptively secure CSS scheme, we first define an intermediate notion of decomposable (computational) secret sharing, and show how it follows from a very simple transformation using any computational secret sharing scheme and symmetric-key encryption. Intuitively, the notion breaks up sharing into two algorithms. The first algorithm is called ShareKey and it generates individual shares $(\text{sh}_1, \dots, \text{sh}_n)$ for each user. Importantly, it produces them *without* knowledge of either the program or the secret to be shared. A second algorithm ShareMsg takes in the program and secret to be shared and creates the public share ps . For this primitive we define and achieve (via our simple transformation) a semi-adaptive notion of security where the attacker can adaptively corrupt individual shares, so long as they have not yet seen the public share.

Syntactically, we require our Share algorithm to be decomposable into two parts:

- $\text{ShareKey}(1^\lambda, 1^n) \rightarrow (\text{sh}_1, \dots, \text{sh}_n), \text{sk}$: On input security parameter λ and number of parties in the access structure n , the first part of the share algorithm distributes the user shares $\text{sh}_1, \dots, \text{sh}_n$ and a secret key sk to be used by the second part of the share algorithm.
- $\text{ShareMsg}(\text{sk}, P, s) \rightarrow \text{ps}$: Using the secret key sk , the program P and secret s , the second part of the share algorithm outputs the public information ps .

We define the correctness and security requirements for decomposable secret sharing as follows:

- **Correctness:** We require our decomposable secret sharing scheme to satisfy the usual CSS notion of (perfect) correctness, taking Share as:
 - $\text{Share}(1^\lambda, P, s)$: On input security parameter λ , program P over n parties, and secret s ,
 - * Sample $(\text{sh}_1, \dots, \text{sh}_n), \text{sk} \leftarrow \text{ShareKey}(1^\lambda, 1^n)$.
 - * Sample $\text{psShareMsg}(\text{sk}, P, s)$.

- * Output $(ps, sh_1, \dots, sh_n)$.

We require our decomposable secret sharing scheme to have a semi-adaptive notion of security. Informally, this gives an adversary the ability to corrupt user shares *before* committing to a challenge policy or secret. The adversary is also given an ‘encryption oracle’ to view secret sharings of other messages under the same ‘key’.

- **Semi-Adaptive Security:** For a bit $b \in \{0, 1\}$ and an adversary $\mathcal{A}$, define the secret sharing security game $\text{ExptSemiSS}_{\mathcal{A}}(\lambda, b)$ as follows:

1. Algorithm $\mathcal{A}(1^\lambda)$ sends a party count 1^n to the challenger, who samples $(sh_1, \dots, sh_n), sk \leftarrow \text{ShareKey}(1^\lambda, 1^n)$
2. The adversary $\mathcal{A}$ can make these queries as much as they want:
 - **Public Share Query:** $\mathcal{A}$ requests a secret s and a program P , and the challenger computes $ps \leftarrow \text{ShareMsg}(sk, P, s)$ and responds with ps .
 - **Corruption Query:** $\mathcal{A}$ requests an index $i \in [n]$, and the challenger responds with sh_i .
3. The adversary $\mathcal{A}$ requests a challenge in the form of two secrets s_0, s_1 and a program P^* . The challenger replies with $ps^* \leftarrow \text{ShareMsg}(sk, P^*, s_b)$.
4. The adversary $\mathcal{A}$ can continue to make queries to the **Public Share Queries** oracle of Step 2 as many times as desired.
5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0.

We say that a decomposable computational secret sharing scheme is semi-adaptively secure if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{ExptSemiSS}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{ExptSemiSS}_{\mathcal{A}}(\lambda, 1) = 1]| = \text{negl}(\lambda).$$

3.1 Construction

We can realize our definition of semi-adaptively secure decomposable secret sharing for any family of access structures using a statically secure CSS for the same family of access structures and symmetric-key encryption.

Construction 3.1 (Decomposable Secret Sharing). We construct a decomposable secret sharing scheme supporting programs $\mathcal{P}$ on secret space $\mathcal{S}$ using the following ingredients:

- $\Pi_S = (S.\text{Share}, S.\text{Recon})$ is a statically secure CSS supporting programs $\mathcal{P}$ on secret space $\mathcal{S}$. Let shsize be the share size of this scheme. Without loss of generality, suppose $\{0, 1\}^{\text{shsize}} \subseteq \mathcal{S}$.
- $\Pi_{\text{SKE}} = (\text{SKE}.\text{KeyGen}, \text{SKE}.\text{Encrypt}, \text{SKE}.\text{Decrypt})$ is a IND-CPA secure symmetric-key encryption scheme supporting message space $\{0, 1\}^{\text{shsize}}$.

Our construction essentially proceeds by encapsulating shares of our static scheme under a symmetric-key encryption scheme. This enables decomposability by allowing us to post the encrypted shares as public information while issuing secret keys as the user shares.

- $\text{ShareKey}(1^\lambda, 1^n)$: On input security parameter 1^λ and number of parties 1^n , the ShareKey algorithm does as follows:
 - For $i \in [n]$, sample secret keys $sk_i \leftarrow \text{SKE}.\text{KeyGen}(1^\lambda)$.
 - Return $\{sh_i = sk_i\}, sk = \{sk_i\}_{i \in [n]}$.
- $\text{ShareMsg}(sk, P, s)$: On input secret key $sk = \{sk_i\}_{i \in [n]}$, program P and secret s , the ShareMsg algorithm does as follows:
 - Compute sharing $(S.ps, S.sh_1, \dots, S.sh_n) \leftarrow S.\text{Share}(1^\lambda, P, s)$.

- For $i \in [n]$, encrypt $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, S.sh_i)$.
- Output $ps = (S.ps, \{ct_i\}_{i \in [n]})$.
- $\text{Recon}(P, X, ps, \{sh_i\}_{i \in X})$: On input policy P , authorized set X , public information ps and shares $\{sh_i\}$, the reconstruction algorithm does as follows:
 - Parse $ps = (S.ps, \{ct_i\}_{i \in [n]})$ and $sh_i = sk_i$.
 - For $i \in X$, recover $S.sh_i \leftarrow \text{SKE.Decrypt}(sk_i, ct_i)$.
 - Run and output $s \leftarrow S.\text{Recon}(P, X, S.ps, \{S.sh_i\}_{i \in X})$.

Lemma 3.2 (Correctness). *Suppose Π_S is a perfectly correct computational secret sharing scheme on some program space $\mathcal{P}$ and secret space $\mathcal{S}$. In addition, suppose Π_{SKE} is a perfectly correct symmetric-key encryption scheme on message space $\{0, 1\}^{\text{shsize}}$, where shsize is the maximum share size of Π_S . Then [Construction 3.1](#) is a perfectly correct decomposable secret sharing scheme on program space $\mathcal{P}$ and secret space $\mathcal{S}$.*

Proof. Recall each share of our scheme sh_i is a secret key for a symmetric-key encryption scheme sk_i . By perfect correctness of Π_{SKE} , $\text{SKE.Decrypt}(sk_i, ct_i)$ will correctly recover the message encrypted under ct_i , which was $S.sh_i$. Now by perfect correctness of Π_S , we can see that $S.\text{Recon}(P, x, S.ps, \{S.sh_i\}_{i \in S_x})$ will correctly recover the secret (recall in both games $P(x) = 1$). $\square$

Lemma 3.3 (Semi-adaptive Security). *Suppose Π_S is a statically secure CSS on some program space $\mathcal{P}$ and secret space $\mathcal{S}$ and Π_{SKE} is an IND-CPA secure symmetric key encryption scheme on message space $\{0, 1\}^{\text{shsize}}$. Then [Construction 3.1](#) is a semi-adaptively secure decomposable secret sharing scheme on program space $\mathcal{P}$ and secret space $\mathcal{S}$.*

Proof. To show semi-adaptive security, we will proceed through a short sequence of hybrids:

- $\text{Hyb}_{\text{real}}^{(b)}$ - This is the real semi-adaptive security game, where the challenger gives a secret sharing of s_b .
 1. Algorithm $\mathcal{A}(1^\lambda)$ sends a party count 1^n to the challenger, who samples $sk_i \leftarrow \text{SKE.KeyGen}(1^\lambda)$ for $i \in [n]$.
 2. The adversary $\mathcal{A}$ can make these queries as much as they want:
 - **Public Share Query:** $\mathcal{A}$ requests a secret s and a program P , which the challenger responds to by
 - * Compute sharing $(S.ps, S.sh_1, \dots, S.sh_n) \leftarrow S.\text{Share}(1^\lambda, P, s)$.
 - * For $i \in [n]$, encrypt $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, S.sh_i)$.
 - * Return $ps = (S.ps, \{ct_i\}_{i \in [n]})$.
 - **Corruption Query:** $\mathcal{A}$ requests an index $i \in [n]$, and the challenger responds with sk_i .
 3. The adversary $\mathcal{A}$ requests a challenge in the form of two secrets s_0, s_1 and a program P^* . The challenger responds by
 - Compute sharing $(S.ps, S.sh_1, \dots, S.sh_n) \leftarrow S.\text{Share}(1^\lambda, P^*, s_b)$.
 - For $i \in [n]$, encrypt $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, S.sh_i)$.
 - Return $(S.ps, \{ct_i\}_{i \in [n]})$.
 4. The adversary $\mathcal{A}$ can continue to make queries to the **Public Share Queries** oracle of Step 2 as many times as desired.
 5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0.
- $\text{Hyb}_\ell^{(b)}$ - In this hybrid, the challenger will switch the ℓ^{th} ciphertext to an encryption of 0 if it has not already been corrupted.

...

3. The adversary $\mathcal{A}$ requests a challenge in the form of two secrets s_0, s_1 and a program P^* . The challenger responds by
 - Compute sharing $(S.ps, S.sh_1, \dots, S.sh_n) \leftarrow S.Share(1^\lambda, P^*, s_b)$.
 - For $i \in [n]$:
 - * If $i \leq \ell$ and $\mathcal{A}$ has not requested a **Corruption Query** on i , encrypt $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, 0^{\text{shsize}})$.
 - * Otherwise encrypt $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, S.sh_i)$.
 - Return $(S.ps, \{ct_i\}_{i \in [n]})$.

Claim 3.4. For all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$,

$$\Pr[\text{Hyb}_{\text{real}}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_0^{(b)}(\mathcal{A}) = 1].$$

Claim 3.5. Suppose Π_{SKE} is an IND-CPA secure symmetric key encryption scheme. Then for all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$, and $\ell \in [n]$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

Claim 3.6. Suppose Π_S is a statically secure computational secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{Hyb}_n^{(0)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_n^{(1)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

We defer the proofs of [Claims 3.4](#) to [3.6](#) to [Appendix B](#).

Taking together [Claims 3.4](#) to [3.6](#) completes the proof of [Lemma 3.3](#). □

4 Constructing Adaptive Secret Sharing

In this section, we build adaptively secure computational secret sharing using the decomposable and semi-adaptive secret sharing we constructed in the previous section. At a high level, our construction works by instantiating $n + 1$ copies of our semi-adaptive scheme and recursively encoding the shares of the j^{th} instantiation of the semi-adaptive scheme as the secrets of the $(j + 1)^{\text{th}}$ instantiation, with the secret being encoded in the last instantiation.

To argue security, we can first leverage semi-adaptive security to remove information on the *last* corruption query made by the adversary. This allows us to replace all shares encoded within another instance of the secret sharing scheme to be replaced with sharings of 0^{shsize} . Now, the last share consists of only one ‘true’ share in the outermost layer, along with n sharings of 0^{shsize} . As a result, we can next examine the second-to-last corruption query, again using semi-adaptive security to erase all but one of the n levels which have had their shares for the last corruption query erased. By continuing to work backwards, we can incrementally erase the shares of the j^{th} level on the last j corruption queries. This allows us to remove *all* shares of the $(n + 1)^{\text{th}}$ level which encodes the message, at which point we can reduce to static security of the $(n + 1)^{\text{th}}$ level.

Construction 4.1. We construct an adaptive computational secret sharing scheme supporting programs $\mathcal{P}$ on secret space $\mathcal{S}$ using the following ingredient:

- $\Pi_{\text{SA}} = (\text{SA.ShareKey}, \text{SA.ShareMsg}, \text{SA.Recon})$ is a semi-adaptively secure decomposable secret sharing scheme supporting programs $\mathcal{P}$ and secret space $\mathcal{S}$. We also require share size shsize of this scheme to be encodeable in the secret space $\{0, 1\}^{\text{shsize}} \subseteq \mathcal{S}$. Since the share size of a CSS scheme can be made independent of the secret space generically with symmetric-key encryption (in fact our transformation [Construction 3.1](#) does this), this does not impose any additional assumptions.

Our construction is as follows:

- $\text{Share}(1^\lambda, P, s)$: On input security parameter 1^λ , policy P (supporting n parties), and secret s , the Share algorithm does as follows:

- For $j \in [0, n]$, compute $(\{sh_i^j\}, sk^j) \leftarrow \text{SA.ShareKey}(1^\lambda, 1^n)$ where sh_i^j is the i^{th} output of the algorithm on iteration j .
 - For $i, j \in [n]$, compute $ps_i^j \leftarrow \text{SA.ShareMsg}(sk^j, P, sh_i^{j-1})$.
 - Compute $T \leftarrow \text{SA.ShareMsg}(sk^0, P, s)$.
 - Output $ps = T, sh_i = (sh_i^n, (ps_i^1, \dots, ps_i^n))^2$.
- **Recon** $(P, X, ps, \{sh_i\}_{i \in X})$: On input policy P , authorized set X , public information ps and shares $\{sh_i\}$ for the set X , the reconstruction algorithm does as follows:
 - Parse $ps = T$ and $sh_i = (sh_i^n, (ps_i^1, \dots, ps_i^n))$.
 - Iterating j from n to 1 and for all $i \in X$:
 - * Recover $sh_i^{j-1} \leftarrow \text{SA.Recon}(P, X, ps_i^j, \{sh_k^j\}_{k \in X})$.
 - Recover and output $s \leftarrow \text{SA.Recon}(P, X, T, \{sh_k^0\}_{k \in X})$.

Remark 4.2. Our transformation preserves the decomposability of the underlying scheme. However, for ease of exposition and definitional clarity, we present this construction with a single Share algorithm.

Theorem 4.3 (Correctness). *Suppose Π_{SA} is a perfectly correct decomposable computational secret sharing scheme. Then [Construction 4.1](#) is a perfectly correct computational secret sharing scheme on the same program and secret space.*

Proof. Let sh_i^{j-1} be the share recovered in the second step of the Recon algorithm. Recall the reconstruction algorithm iteratively computes this as $\text{SA.Recon}(P, X, ps_i^j, \{sh_k^j\}_{k \in X})$. We can inductively argue that this is identical to sh_i^{j-1} computed as per Share for all $i \in X, j \in [n]$. First, observe that when $j = n$, this follows trivially as sh_i^n is given out in the clear in sh_i . Now, suppose that $sh_k^j = sh_k^j$ for all $k \in X$. By perfect correctness of Π_{SA} , this successfully recovers sh_i^{j-1} as it was shared in Share for all $i \in X$. Applying this inductively, we can conclude that Recon recovers sh_i^0 for all $i \in X$, so by perfect correctness of Π_{SA} , $\text{SA.Recon}(P, X, T, \{sh_k^0\}_{k \in X})$ will correctly reconstruct s . $\square$

Theorem 4.4 (Adaptive Security). *Suppose Π_{SA} is a semi-adaptively secure decomposable computational secret sharing scheme. Then [Construction 4.1](#) is an adaptively secure computational secret sharing scheme on the same program and secret space.*

Proof. To show adaptive security, we will proceed through a sequence of hybrids.

- $\text{Hyb}_{\text{real}}^{(b)}$ - This is the real adaptive security game, where the challenger gives a secret sharing of s_b .
 1. Algorithm $\mathcal{A}(1^\lambda)$ sends a program P on n inputs and a pair of secrets s_0, s_1 to the challenger, who samples a Share program.
 - For $j \in [0, n]$, compute $(\{sh_i^j\}, sk^j) \leftarrow \text{SA.ShareKey}(1^\lambda, 1^n)$ where sh_i^j is the i^{th} output of the algorithm on iteration j .
 - For $i, j \in [n]$, compute $ps_i^j \leftarrow \text{SA.ShareMsg}(sk^j, P, sh_i^{j-1})$.
 - Compute $T \leftarrow \text{SA.ShareMsg}(sk^0, P, s_b)$.
 - Return $ps = T$ to the $\mathcal{A}$.
 2. For as many times as desired, the adversary $\mathcal{A}$ issues query i to the challenger. The challenger responds with $(sh_i^n, (ps_i^1, \dots, ps_i^n))$.
 3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of parties for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0.

²Recall as in the intro ps_i^j takes the form of a *public share* for the underlying semi-adaptive scheme but is part of the *user share* of the adaptive scheme

- $\text{Hyb}_\ell^{(b)}$ - For $\ell \in [n]$, in this hybrid, the challenger will switch the way they sample the $(n - \ell + 1)^{th}$ corruption query. Here, they will sample the last ℓ ‘subshares’ as normal and replace all the others with sharings of 0.

...

2. For as many times as desired, the adversary $\mathcal{A}$ issues corruption queries to the challenger. For the k^{th} corruption query (on party i_k):

- If $k < (n - \ell + 1)$, compute sample sh_{i_k} as in $\text{Hyb}_{\text{real}}^{(b)}$.
- Otherwise for $j \in [n]$:
 - * For $j > k$ set $\text{ps}_{i_k}^j$ as before.
 - * Otherwise, for $j \leq k$, instead set as $\text{ps}_{i_k}^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, 0^{\text{shsize}})$.
 - * Output $(\text{sh}_{i_k}^n, (\text{ps}_{i_k}^1, \dots, \text{ps}_{i_k}^n))$.

...

Lemma 4.5. For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$,

$$\Pr[\text{Hyb}_{\text{real}}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_0^{(b)}(\mathcal{A}) = 1]$$

Proof. Since P supports n parties, the adversary can make at most n unique corruption queries. Since k is always $< n + 1$, these are games identical. $\square$

Lemma 4.6. Suppose Π_{SA} is a semi-adaptively secure decomposable secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$ and $\ell \in [n]$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

Proof. First, note that if the adversary makes less than $n - \ell + 1$ corruption queries, the adversary’s view of these hybrids is identical. Thus, we will focus on the case the adversary makes at least $n - \ell + 1$ corruption queries. To show this indistinguishability, we will proceed through a sequence of subhybrids $\text{sHyb}_{\ell-1, k^*}^{(b)}$ ranging $k^* \in [0, n - \ell + 1]$ between $\text{Hyb}_{\ell-1}^{(b)}$ and $\text{Hyb}_\ell^{(b)}$, defined as follows:

- $\text{sHyb}_{\ell-1, k^*}^{(b)}$ - This hybrid proceeds as $\text{Hyb}_{\ell-1}^{(b)}$, except on the $k = (n - \ell + 1)^{th}$ corruption query (corrupting party i_k), the adversary will replace $\text{ps}_{i_k}^j$ with a sharing of 0^{shsize} for $j \in [k^*]$.

...

2. For as many times as desired, the adversary $\mathcal{A}$ issues Query i to the challenger. For the k^{th} corruption query (on party i_k):

- If $k \neq n - \ell + 1$, compute sample sh_i as in $\text{Hyb}_{\ell-1}^{(b)}$.
- Otherwise (when $k = n - \ell + 1$), for $j \in [n]$:
 - * For $j \in [k^*]$, set $\text{ps}_{i_k}^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, 0^{\text{shsize}})$.
 - * Otherwise, keep $\text{ps}_{i_k}^j$ as before.
 - * Output $(\text{sh}_{i_k}^n, (\text{ps}_{i_k}^1, \dots, \text{ps}_{i_k}^n))$.

...

Claim 4.7. For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$ and $\ell \in [n]$,

$$\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{sHyb}_{\ell-1, 0}^{(b)}(\mathcal{A}) = 1].$$

Proof. Note when $k^* = 0$, the set $[0] = \emptyset$, and so $\text{sHyb}_{\ell-1, 0}^{(b)}$ is identical to $\text{Hyb}_{\ell-1}^{(b)}$. $\square$

Claim 4.8. Suppose Π_{SA} is a semi-adaptively secure decomposable secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$, $\ell \in [n]$, and $k^* \in [n - \ell + 1]$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{sHyb}_{\ell-1, k^*-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{sHyb}_{\ell-1, k^*}^{(b)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

Proof. Suppose that there exists an efficient adversary $\mathcal{A}$ that can distinguish these two experiments with non-negligible probability ε . We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that breaks security of Π_{SA} with the same advantage:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per $\text{sHyb}_{\ell-1, k^*}^{(b)}$.
1. Algorithm $\mathcal{A}(1^\lambda)$ sends a program P and a pair of secrets s_0, s_1 to $\mathcal{B}$, who computes the following:
 - For $j \in [0, n] \setminus \{k^*\}$, compute $(\{\text{sh}_i^j\}, \text{sk}^j) \leftarrow \text{SA.ShareKey}(1^\lambda, 1^n)$ where sh_i^j is the i -th output of the algorithm on iteration j .
 - For $i \in [n], j \in [n] \setminus \{k^*\}$, compute $\text{ps}_i^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, \text{sh}_i^{j-1})$.
 - Compute $T \leftarrow \text{SA.ShareMsg}(\text{sk}^0, P, s_b)$.
 - Return $\text{ps} = T$ to the $\mathcal{A}$.
2. For as many times as desired, the adversary $\mathcal{A}$ makes corruption queries to $\mathcal{B}$. On the k^{th} corruption query to party i_k , $\mathcal{B}$ computes response $(\text{sh}_{i_k}^n, (\text{ps}_{i_k}^1, \dots, \text{ps}_{i_k}^n))$ as follows:
 - (a) For $j \neq \{k^*, k^* + 1\}$, since the reduction $\mathcal{B}$ has sampled both sk^j and $\text{sh}_{i_k}^{j-1}$, $\mathcal{B}$ sets $\text{ps}_{i_k}^j$ as $\text{sHyb}_{\ell-1, k^*-1}^{(b)}$.
 - (b) To compute $\text{ps}_{i_k}^{k^*+1}$, recall the reduction $\mathcal{B}$ did not sample $\text{sh}_{i_k}^{k^*}$, and so:
 - i. If $k \leq (n - \ell + 1)$, $\mathcal{B}$ submits a **Corruption Query** to the Semi-Adaptive CSS challenger on party i_k , receiving response $\text{sh}_{i_k}^{k^*}$. $\mathcal{B}$ uses this to set $\text{ps}_{i_k}^{k^*+1} \leftarrow \text{SA.ShareMsg}(\text{sk}^{k^*+1}, P, \text{sh}_{i_k}^{k^*})$ ³.
 - ii. For $k > (n - \ell + 1)$, $\mathcal{B}$ instead computes $\text{ps}_{i_k}^{k^*+1} \leftarrow \text{SA.ShareMsg}(\text{sk}^{k^*+1}, P, 0^{\text{shsize}})$.
 - (c) To compute $\text{ps}_{i_k}^{k^*}$, recall the reduction $\mathcal{B}$ did not sample sk^{k^*} , and so:
 - i. If $k < (n - \ell + 1)$, submit **Public Share Query** on policy P and secret $\text{sh}_i^{k^*-1}$, receiving response $\text{ps}_{i_k}^{k^*}$.
 - ii. If $k = (n - \ell + 1)$, $\mathcal{B}$ submits **Challenge Query** on policy P , set X^* , and secrets $s_0 = \text{sh}_i^{k^*-1}, s_1 = 0^{\text{shsize}}$, receiving challenge response $\text{ps}_{i_k}^{k^*}$.
 - iii. If $k > (n - \ell + 1)$, submit **Public Share Query** on policy P and secret 0^{shsize} , receiving response $\text{ps}_{i_k}^{k^*}$.
3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of parties $\subseteq [n]$ for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0. $\mathcal{B}$ returns the output of the experiment.

The validity of our reduction here relies on the observation that:

- $\text{sHyb}_{\ell-1, k^*-1}^{(b)}$ and $\text{sHyb}_{\ell-1, k^*}^{(b)}$ differ only on the value of $\text{ps}_i^{k^*}$, as encodings of sh_i^ℓ or 0^{shsize} respectively (where i is the $(n - \ell + 1)^{\text{th}}$ corrupted party).
- On corruption queries k after the $(n - \ell + 1)^{\text{th}}$, since $\text{ps}_{i_k}^{k^*+1}$ has already been switched to sharings of 0^{shsize} , the reduction never has to give out share keys for $\text{sh}_{i_k}^{k^*}$. Thus, by the time $\mathcal{B}$ issues a challenge query in step (c).ii, it has finished making all the corruption queries on step (b).i. Thus, the *only* parties for which we need to give share keys on the $(k^* + 1)^{\text{th}}$ level are fixed by Corruption Query $(n - \ell + 1)$, allowing to leverage semi-adaptivity.

Taking these two together, we can see that $\mathcal{B}$ is a valid semi-adaptive adversary which simulates $\text{sHyb}_{\ell-1, k^*-1}^{(b)}$ when $b = 0$ and $\text{sHyb}_{\ell-1, k^*}^{(b)}$ when $b = 1$. $\square$

³When $k^* = n$, we simply give out $\text{sh}_{i_k}^{k^*}$ in the clear.

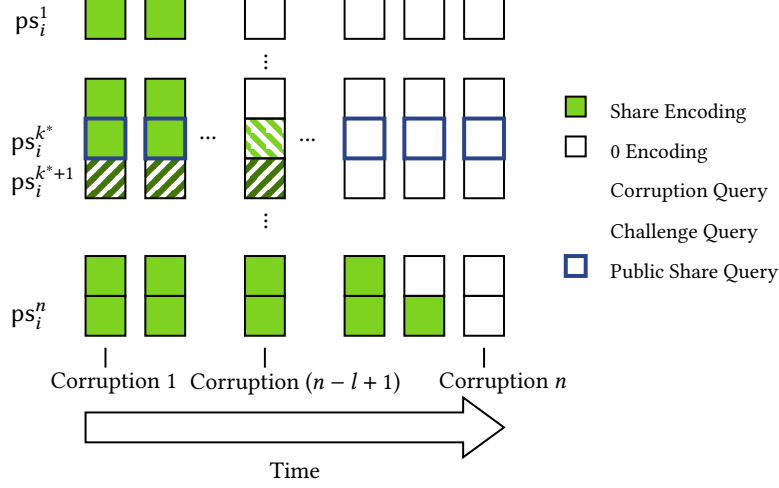


Figure 2: Structure of how shares are simulated in reduction of Claim 4.8.

Claim 4.9. For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$ and $\ell \in [n]$,

$$\Pr[\text{sHyb}_{\ell-1, n-\ell+1}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1].$$

Proof. Observe here, on the $k = (n - \ell + 1)^{\text{th}}$ corruption query, $ps_{i_k}^j$ is set to $\text{SA.ShareMsg}(\text{sk}^j, P, 0^{\text{shsize}})$ for $j \in [k^*] = [k]$. This is equivalent to the check that $j \geq k$ in $\text{Hyb}_\ell^{(b)}$. $\square$

Taking together Claims 4.7 to 4.9, we get the proof of our Lemma 4.6. $\square$

Lemma 4.10. Suppose Π_{SA} is a semi-adaptively secure decomposable secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$,

$$|\Pr[\text{Hyb}_n^{(0)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_n^{(1)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

Proof. Observe in $\text{Hyb}_n^{(b)}$, because $k < (n - \ell + 1) = 1$ never occurs, the k^{th} corruption query (on party i_k) is always generated as follows:

- For $j > k$ set $ps_{i_k}^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, \text{sh}_{i_k}^{j-1})$.
- For $j \leq k$ set $ps_{i_k}^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, 0^{\text{shsize}})$.
- Output $(\text{sh}_{i_k}^n, (ps_{i_k}^1 \dots ps_{i_k}^n))$.

Moreover, since k is at least 1, we always encode ps_i^1 as a sharing of 0^{shsize} . Thus, the challenger never gives any out any shares of sh_i^0 (apart from the public information T). Since this is the only sharing dependent on secret s , this can be reduced to the semi-adaptive (and in fact static) security of Π_{SA} . We formalize this observation in the reduction $\mathcal{B}$ below:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per $\text{Hyb}_n^{(b)}(\mathcal{A})$.
1. Algorithm $\mathcal{A}(1^\lambda)$ sends a program P and a pair of secrets s_0, s_1 to $\mathcal{B}$, who computes the following:
 - For $j \in [n]$, compute $(\{\text{sh}_i^j\}, \text{sk}^j) \leftarrow \text{SA.ShareKey}(1^\lambda, 1^n)$.
 - For $i \in [n], j \in [2, n]$, compute $ps_i^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, \text{sh}_i^{j-1})$.
 - For $i \in [n]$, compute $ps_i^1 \leftarrow \text{SA.ShareMsg}(\text{sk}^1, P, 0^{\text{shsize}})$

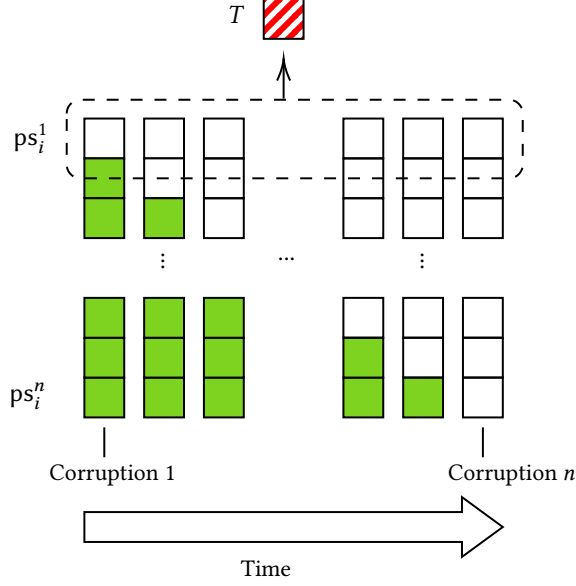


Figure 3: Structure of shares in $\text{Hyb}_n^{(b)}$. Note ps_i^1 has entirely been replaced with sharings of the 0 string, allowing us to argue indistinguishability of public information T in [Claim 4.9](#).

- $\mathcal{B}$ requests a challenge query from the semi-adaptive CSS challenger on secrets s_0, s_1 and program P , receiving T .
 - Return $ps = T$ to the $\mathcal{A}$.
3. $\mathcal{B}$ simulates corruption queries and the challenge exactly as in $\text{Hyb}_n^{(b)}$.
 4. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of parties for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0. $\mathcal{B}$ returns the output of the experiment.

Note this is a secret sharing challenger which never issues any corruption queries, and moreover embeds the secret s_b as the corresponding secret of $\text{Hyb}_n^{(b)}$. □

Taking together [Lemmas 4.5, 4.6](#) and [4.10](#), we get the adaptive security of our construction. □

As an immediate result of our transformation, we obtain constructions of adaptively secure computational secret sharing from weaker assumptions than previously known.

Corollary 4.11 ([Yao89, VNS⁺03]). *Assuming polynomially secure one way functions, there exists adaptively secure secret sharing for monotone boolean circuits policies.*

Corollary 4.12 ([KNY14]). *Assuming polynomially secure witness encryption and one way functions, there exists adaptively secure secret sharing for monotone NP policies.*

5 Direct Proofs of Adaptivity for Simple Policy Families

Our main result of [Section 4](#) had the advantage that it worked for any policy family including quite expressive ones such as monotone boolean circuits. In this section we observe that for some more basic (and limited) policy families it is possible to get more direct proofs of adaptive security that will have less overhead.

We give two such examples. We first show a transformation for any threshold policy. Next we prove that the succinct computational sharing scheme for graph policies of [ABI⁺23] is adaptively secure. Both of these proofs follow rather straightforwardly from a technical perspective. Nonetheless we include them to demonstrate the potential utility of leveraging properties of a particular family of access structure to proving adaptive security even more efficiently.

5.1 Simpler Adaptivity for Threshold Policies

In this section, we present a simple generic static to adaptive transformation for computational secret sharing on threshold policies. Although information-theoretic secret sharing is known for threshold policies [Sha79, Bla79], computational assumptions are commonly utilized for either improved efficiency [Kra93] or to support enhanced functionality (e.g. homomorphic secret sharing [BGI16, BGI⁺18]) or security (e.g. verifiable secret sharing [Fel87, Ped91]). This transformation gives adaptive security only an additive $O(\log n)$ overhead in share size. We do caution, however, that some properties such as additive reconstruction might be lost when applying our transformations.

Construction 5.1. We present a transformation from a statically secure CSS $\Pi_S = (\text{S.Share}, \text{S.Recon})$ on threshold policies to an adaptively secure secret one on the same secret space $\mathcal{S}$. Our transformation simply permutes the underlying shares of the statically secure scheme. This allows us to map the set of shares an adversary adaptively corrupts to a predetermined random subset of shares. Since in a threshold access policy, these shares are functionally identical, this remapping does not affect functionality, and allows our reduction to corrupt any arbitrary $t - 1$ share of the underlying scheme.

- $\text{Share}(1^\lambda, P, s)$: On input security parameter 1^λ , policy $P = (n, t)$ (representing a t -of- n threshold), and secret s , the Share algorithm:
 - Sample a random permutation on n elements $\sigma : [n] \rightarrow [n]$.
 - Compute shares $\text{S.ps}, (\text{S.sh}_1, \dots, \text{S.sh}_n) \leftarrow \text{S.Share}(1^\lambda, (n, t), s)$.
 - Output $\text{ps} = \text{S.ps}, \{\text{sh}_i = (\sigma(i), \text{S.sh}_{\sigma(i)})\}_{i \in [n]}$
- $\text{Recon}(P, X, \text{ps}, \{\text{sh}_i\}_{i \in X})$: On input policy $P = (n, t)$, authorized set X , public information ps and shares $\{\text{sh}_i\}$ for the set X , the reconstruction algorithm:
 - Parses each share as $\text{sh}_i = (j, \text{S.sh}_j)$.
 - Define $X' \in 2^{[n]}$ to be $\{j \mid \text{sh}_i = (j, \text{S.sh}_j) : i \in X\}$.
 - Compute and output $s \leftarrow \text{S.Recon}(P, X', \{\text{S.sh}_j\}_{j \in X'})$

Lemma 5.2 (Correctness). *Suppose Π_S is a perfectly correct CSS scheme. Then Construction 5.1 is perfectly correct computational secret sharing scheme.*

Proof. We can observe that the reconstruction algorithm of Construction 5.1 simply calls S.Recon on the set of shares $\{\text{S.sh}_{\sigma(i)}\}_{i \in X}$. Since σ is a permutation, this set is of the same size as $\{\text{sh}_i\}_{i \in X}$. Thus, if x is an authorized input (i.e. has at least t 1's), then X' must be as well, so correctness follows from perfect correctness of Π_S . $\square$

Lemma 5.3 (Adaptive Security). *Suppose Π_S is a statically secure CSS scheme. Then Construction 5.1 is an adaptively secure CSS scheme.*

Proof. Suppose that there exists an efficient adversary $\mathcal{A}$ that can break the adaptive security of our construction with non-negligible probability ϵ . We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that breaks security of Π_S with the same advantage. Without loss of generality, we assume $\mathcal{A}$ requests *exactly* $t - 1$ corruption queries. We can see with a threshold policy, any valid adversary which queries on less than $t - 1$ parties can be extended by querying arbitrary new parties and simply ignoring them.

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the adaptive CSS challenger.

1. Adversary $\mathcal{A}$ sends a policy $P = (n, t)$ and a pair of secrets s_0, s_1 to $\mathcal{B}$, who computes the following:
 - $\mathcal{B}$ samples $t - 1$ random elements $X^* = (j_1, \dots, j_{t-1}) \subseteq [n]$ without replacement.
 - $\mathcal{B}$ sends the policy (n, t) , input X^* , and secrets s_0, s_1 to the static CSS challenger, and receives $S.ps, \{S.sh_i\}_{i \in X^*}$.
 - $\mathcal{B}$ returns $ps = S.ps$ to the adversary $\mathcal{A}$.
2. For $t - 1$ times, $\mathcal{A}$ sends corruption queries. On the k^{th} query on index i_k , $\mathcal{B}$ responds by returning $sh_{i_k} = (j_k, S.sh_{j_k})$.
3. Adversary $\mathcal{A}$ outputs a bit b' , which $\mathcal{B}$ forwards to the challenger.

We can see that $\mathcal{B}$ is a valid adversary in the static CSS game, as they only query $t - 1$ parties. Furthermore, we can observe that for any set of corruption queries $i_1, \dots, i_{t-1}$, since σ is a uniformly random permutation, the distribution of $(\sigma(i_1), \dots, \sigma(i_{t-1}))$ is identical to a uniform size $t - 1$ subset of $[n]$, even conditional on $i_1, \dots, i_{t-1}$. Thus, this is distributed exactly as set X^* generated by $\mathcal{B}$, so $\mathcal{B}$ faithfully simulates the adaptive CSS game for adversary $\mathcal{A}$. Thus, if $\mathcal{A}$ is an adversary which can win the adaptive CSS game on [Construction 5.1](#) with nonnegligible probability ϵ , then $\mathcal{B}$ can win the static CSS game on Π_S with the same probability. $\square$

5.2 Succinct Adaptive CSS for Graph Policies

In this section, we examine the construction of succinct CSS of [\[ABI⁺23\]](#). In their work, they leverage a new primitive, projective PRGs, to achieve compress the shares sizes of secret sharing schemes for various policy families, giving the first instantiations of *succinct* computational secret sharing (SCSS) for many function families. However, like most prior work, they only consider a static definition of security. Given the longstanding barriers in representation size for secret sharing, as well as known impossibilities in combining adaptive security with succinctness in other cryptographic settings (e.g. functional encryption [\[AGVW13, AKW18\]](#)), a natural question would be the feasibility of succinct and adaptively secure CSS. Although our [Section 4](#) transformation can be applied to said CSS schemes without introducing additional security assumptions, the overhead it incurs loses the succinctness advantages for some policies such as the construction specialized for graphs.⁴ Nevertheless, for some policies we show that adaptively secure SCSS can be realized directly. As a case study, we look at the [\[ABI⁺23\]](#) construction of succinct CSS for graph access policies, and show that it is adaptively secure.

Definition 5.4 (Projective PRG [\[ABI⁺23\]](#)). Let λ be a security parameter. A projective pseudorandom generator (pPRG) is a tuple of efficient randomized algorithms $\Pi_{pPRG} = (\text{Setup}, \text{KeyGen}, \text{Eval})$:

- $\text{Setup}(1^\lambda, 1^m) \rightarrow pp, \text{msk}$: On input security parameter λ and output length m , the setup algorithm outputs public parameters pp and master secret key msk .
- $\text{KeyGen}(pp, \text{msk}, T) \rightarrow \text{sk}_T$: On input public parameters pp , master secret msk , and a set $T \subseteq [m]$, the key generation algorithm outputs a projective key $\text{sk}\{T\}$.
- $\text{Eval}(pp, \text{sk}\{T\} \cup \{\text{msk}\}, T) \rightarrow y = \{y_i\}_{i \in T}$: On input public parameters pp , projective key $\text{sk}\{T\}$, and set T , the evaluation algorithm deterministically outputs a string $y \in \{0, 1\}^{|T|}$, the value of the projective PRG on the bits in T . For indexing convenience, we will sometimes refer to this as a set of bits $\{y_i\}_{i \in T}$. We also permit the Eval algorithm to take in the master public key msk , in which case the projected set is the entire output space $[m]$.

We require the following properties:

- **Correctness:** For all security parameters λ , output lengths m , pp, msk in the support of $\text{Setup}(1^\lambda, 1^m)$, subsets $T \subseteq [m]$ and projective keys $\text{sk}\{T\}$ in the support of $\text{KeyGen}(pp, \text{msk}, T)$, it holds that

$$y = c[T]$$

where y, c are the outputs of $\text{Eval}(pp, \text{sk}\{T\}, T)$ and $\text{Eval}(pp, \text{msk}, [m])$ respectively, and $c[T]$ is substring of c corresponding to the elements of T .

⁴In other cases such as their monotone circuit construction, there is some additional overhead; however some succinctness benefits are maintained after the transformation.

- **Succinctness:** Weak succinctness requires that there is a constant > 0 such that for every $T \subseteq [m]$, the bit-length of $\text{sk}\{T\}$ is asymptotically sublinear in m ($m^{1-\delta} \cdot \text{poly}(\lambda)$). Strong succinctness requires that $\text{sk}\{T\}$ is polylogarithmic in m - i.e. $\text{poly}(\log m, \lambda)$.

- **Security:** For a bit $b \in \{0, 1\}$ and an adversary $\mathcal{A}$, the robustness experiment $\text{ExptPPRG}_{\mathcal{A}}(\lambda, b)$ is as follows:

1. The adversary $\mathcal{A}$ selects an output length 1^m and sets $T_1, T_2, \dots, T_\ell \subseteq [m]$ which they send to the challenger
2. The challenger samples $\text{pp}, \text{msk} \leftarrow \text{Setup}(1^\lambda, 1^m)$ and $\text{sk}\{T_i\} \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, T_i)$ for $i \in [\ell]$. The challenger also computes

$$T^* = [m] \setminus \bigcup_{i \in [\ell]} T_i,$$

and evaluates $\text{sk}\{T^*\} \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, T^*)$. They compute challenge strings $c_0 \xleftarrow{\mathbb{R}} \{0, 1\}^{|T^*|}$ and $c_1 \leftarrow \text{Eval}(\text{pp}, \text{sk}\{T^*\}, T^*)$, and sends the adversary $(\text{pp}, \{\text{sk}\{T_i\}\}_{i \in [\ell]}, c_b)$.

3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$, which is the output of the experiment.

We require that for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{ExptPPRG}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{ExptPPRG}_{\mathcal{A}}(\lambda, 1) = 1]| = \text{negl}(\lambda).$$

Remark 5.5. We note that the security definition we provide above is was previously referred to [ABI⁺23] as *robustness*. Applebaum et. al. [ABI⁺23] also considers a weaker notion of projective PRG security where the adversary only sees one projective key. However, since their constructions all satisfy this stronger definition, we will omit the weaker security definition.

Definition 5.6 (Graph Access Policies [BD89]). A graph access policy is represented by an undirected graph $G = (V, E)$ where the access policy is over the set of parties V and a set V' is authorized iff $\exists v_0, v_1 \in V' : (v_0, v_1) \in E$.

Construction 5.7 ([ABI⁺23]). We recall the construction of graph secret sharing [ABI⁺23] from a projective PRG scheme $\Pi_{\text{pPRG}} = (\text{pPRG.Setup}, \text{pPRG.KeyGen}, \text{pPRG.Eval})$. As with their presentation, we will only consider one bit secrets. As observed in [BFM12], it suffices to consider bipartite graph policies.

- **Share**($1^\lambda, P, s$) : The Share algorithm takes as input security parameter 1^λ , policy $P = (L, R, E)$ and secret $s \in \{0, 1\}$. We will use $N(v)$ to refer to the neighborhood of a vertex v . The algorithm computes:
 - Sample $(\text{pp}, \text{msk}) \leftarrow \text{pPRG.Setup}(1^\lambda, 1^{|L|})$.
 - Compute $\{c_u\}_{u \in L} \leftarrow \text{pPRG.Eval}(\text{pp}, \text{msk})$.
 - For $u \in L$, set $\text{sh}_u = s \oplus c_u$.
 - For $v \in R$, set $\text{sh}_v = \text{pPRG.KeyGen}(\text{pp}, \text{msk}, N(v))$.
 - Output $\text{ps} = \text{pp}, \{\text{sh}_v\}_{v \in L \cup R}$.
- **Recon**($P, X, \text{ps}, \{\text{sh}_i\}_{i \in X}$) : The Recon algorithm takes in a policy $P = (L, R, E)$, an authorized set X , public information $\text{ps} = \text{pp}$, and a set of shares $\{\text{sh}_i\}_{i \in X}$. Recall the reconstruction set X is authorized iff it contains an edge $(u^*, v^*) \in E$. Since our graph is bipartite, without loss of generality, we assume $u \in L, v \in R$. Our algorithm recovers the secret as:
 - Parse $\text{sh}_{u^*} = c'_{u^*}$ and $\text{sh}_{v^*} = \text{sk}\{N(v^*)\}$.
 - Compute $\{c_u\}_{u \in N(v^*)} \leftarrow \text{pPRG.Eval}(\text{pp}, \text{sk}\{N(v^*)\}, N(v^*))$.
 - Output $c'_{u^*} \oplus c_{u^*}$.

We refer the reader to Section 4.1 of [ABI⁺23] for the correctness and additional discussion on this scheme, and will simply argue adaptive security.

Lemma 5.8 (Adaptive Security). *Suppose Π_{pPRG} is a secure projective PRG. Then [Construction 5.7](#) is an adaptively secure computational secret sharing scheme.*

Proof. To show adaptive security, we will proceed through a short sequence of hybrids:

- $\text{Hyb}_{\text{real}}^{(b)}$ - This is the real adaptive security game, where the challenger gives a secret sharing of the bit b .
 1. Algorithm $\mathcal{A}(1^\lambda)$ sends a program $P = (L, R, E)$ to the challenger, who samples $\text{pp}, \text{msk} \leftarrow \text{pPRG.Setup}(1^\lambda, 1^{|L|})$, computes $\{c_v\}_{v \in L} \leftarrow \text{pPRG.Eval}(\text{pp}, \text{msk})$ and returns $\text{ps} = \text{pp}$ to $\mathcal{A}$.
 2. For as many times as desired, the adversary $\mathcal{A}$ issues query v to the challenger.
 - If $v \in L$, the challenger returns $\text{sh}_v = c_v \oplus b$.
 - If $v \in R$, the challenger returns $\text{sh}_v = \text{pPRG.KeyGen}(\text{pp}, \text{msk}, N(v))$.
 3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let $C = (L_C, R_C)$ be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $\nexists u \in L_C, v \in R_C : (u, v) \in E$, then the output of the experiment is b' . Otherwise the experiment outputs 0.
- $\text{Hyb}_\ell^{(b)}$ - In this hybrid, the challenger removes the secret from the shares of the first ℓ left vertices. For notational convenience, we will assume the left vertices are indexed by the set $[|L|]$.
 2. For as many times as desired, the adversary $\mathcal{A}$ issues query v to the challenger.
 - If $v \in L$ and $v \leq \ell$ the challenger returns $\text{sh}_v = c_v$. For other $v \in L$, the challenger returns $\text{sh}_v = c_v \oplus s$.
 - If $v \in R$, the challenger returns $\text{sh}_v = \text{pPRG.KeyGen}(\text{pp}, \text{msk}, N(v))$.
 3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let $C = (L_C, R_C)$ be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $\nexists u \in L_C, v \in R_C : (u, v) \in E$, then the output of the experiment is b' . Otherwise the experiment outputs 0.

Claim 5.9. *For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$,*

$$\Pr[\text{Hyb}_{\text{real}}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_0^{(b)}(\mathcal{A}) = 1].$$

Proof. Since $v \leq 0$ is never true, these hybrids are identical. $\square$

Claim 5.10. *Suppose Π_{pPRG} is a secure projective PRG. Then for all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$, and $\ell \in [n]$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,*

$$|\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

Proof. To analyze these hybrids, we will consider two cases based on if the event E_ℓ that the adversary makes a corruption query on vertex ℓ . We can show in both cases, the adversary's distinguishing advantage between these two hybrids is negligible. First, it is easy to see that when E_ℓ does not occur, the adversary never receives sh_ℓ , and thus their view is *independent* of the distribution of sh_ℓ . Since this is the only difference between $\text{Hyb}_{\ell-1}^{(b)}$ and $\text{Hyb}_\ell^{(b)}$, we can conclude the adversary's advantage in this case is 0. Thus, it remains to show that

$$|\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | E_\ell] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | E_\ell]| = \text{negl}(\lambda).$$

Here, we argue that rely on pPRG security to argue that sh_ℓ is indistinguishable from a random bit. We formalize this in the following intermediate hybrid iHyb_ℓ :

...

2. For as many times as desired, the adversary $\mathcal{A}$ issues query v to the challenger.
 - If $v \in L$ and $v < \ell$ the challenger returns $\text{sh}_v = c_v$. For other $v \in L$ and $v > \ell$, the challenger returns $\text{sh}_v = c_v \oplus s$. **The challenger samples $c_\ell \xleftarrow{R} \{0, 1\}$.**

- If $v \in R$, the challenger returns $\text{sh}_v = \text{pPRG.KeyGen}(\text{pp}, \text{msk}, N(v))$.
3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let $C = (L_C, R_C)$ be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $\nexists u \in L_C, v \in R_C : (u, v) \in E$, then the output of the experiment is b' . Otherwise the experiment outputs 0.

Suppose an adversary $\mathcal{A}$ can distinguish between $\text{Hyb}_{\ell-1}^{(b)}$ and iHyb_ℓ with noticeable probability. We will use $\mathcal{A}$ to construct a reduction $\mathcal{B}$ that breaks security of Π_{pPRG} with the same advantage:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per $\text{Hyb}_{\ell-1}^{(b)}$.
1. Algorithm $\mathcal{A}(1^\lambda)$ sends a program $P = (L, R, E)$ to $\mathcal{B}$, who queries the pPRG challenger
 - Define the set R_ℓ to be the subset of R such that $v \in R_\ell \Leftrightarrow \ell \notin N(v)$.
 - Algorithm $\mathcal{B}$ select output length $1^{|L|}$ and sets $\{N(v) \mid v \in R_\ell\}, \{[m] \setminus \ell\}$ to the pPRG challenger.
 - $\mathcal{B}$ receives public parameters pp , projective PRG keys $\{\text{sk}\{N(v)\}\}_{v \in R_\ell}, \text{sk}\{[m] \setminus \ell\}$, and challenge bit c_ℓ^* .
 - Compute $\{c_v\}_{v \in [m] \setminus \ell} \leftarrow \text{Eval}(\text{pp}, \text{sk}\{[m] \setminus \ell\}, [m] \setminus \ell)$.
2. For as many times as desired, the adversary $\mathcal{A}$ issues query v to the challenger.
 - If $v \in L$ and $v < \ell$ the challenger returns $\text{sh}_v = c_v$. For other $v > \ell$, the challenger returns $\text{sh}_v = c_v \oplus s$. On sh_ℓ , return $c_\ell^* \oplus s$
 - If $v \in R$, the challenger returns $\text{sh}_v = \text{sk}\{N(v)\}$ received from the pPRG challenger.
3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let $C = (L_C, R_C)$ be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $\nexists u \in L_C, v \in R_C : (u, v) \in E$, then the output of the experiment is b' . Otherwise the experiment outputs 0.

Observe that when c_ℓ^* is the pPRG evaluation, this is exactly $\text{Hyb}_{\ell-1}^{(b)}$, and when c_ℓ^* is a random bit, this is exactly iHyb_ℓ , since a random bit $\oplus s$ is still uniformly random. In addition, observe that since E_ℓ does occur -i.e. the adversary makes a corruption query on vertex ℓ , and so for $\mathcal{A}$ to be a valid adversary, they only corrupt vertices $\in R$ not adjacent to ℓ . These correspond exactly to all the projective keys $\mathcal{B}$ requested from the pPRG challenger.

We can analogously argue the indistinguishability of iHyb_ℓ and $\text{Hyb}_\ell^{(b)}$, which, together, gives us the proof of [Claim 5.10](#). □

Claim 5.11. For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$,

$$\Pr[\text{Hyb}_{|L|}^{(0)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_{|L|}^{(1)}(\mathcal{A}) = 1].$$

Proof. We can see once $\ell = |L|$, left shares are exclusively encoded as c_v . Hence, the adversary's view is independent of the secret. □

Taking together [Claims 5.9](#) to [5.11](#), we get the proof of [Lemma 5.8](#). □

Acknowledgments

Brent Waters is supported by NSF CNS-2318701 and a Simons Investigator award.

References

- [ABI⁺23] Benny Applebaum, Amos Beimel, Yuval Ishai, Eyal Kushilevitz, Tianren Liu, and Vinod Vaikuntanathan. Succinct computational secret sharing. In *STOC*, 2023.
- [AGVW13] Shweta Agrawal, Sergey Gorbunov, Vinod Vaikuntanathan, and Hoeteck Wee. Functional encryption: New perspectives and lower bounds. In *CRYPTO*, 2013.
- [AKW18] Shashank Agrawal, Venkata Koppula, and Brent Waters. Impossibility of simulation secure functional encryption even with random oracles. In *TCC*, 2018.
- [BD89] Ernest F. Brickell and Daniel M. Davenport. On the classification of ideal secret sharing schemes. In *CRYPTO*, 1989.
- [Bei96] Amos Beimel. *Secure Schemes for Secret Sharing and Key Distribution*. PhD thesis, Technion, 1996.
- [BFM12] Amos Beimel, Oriol Farràs, and Yuval Mintz. Secret sharing schemes for very dense graphs. In *CRYPTO*, 2012.
- [BGI16] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In *CRYPTO*, 2016.
- [BGI⁺18] Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. In *ITCS*, 2018.
- [BL88] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In *CRYPTO*, 1988.
- [Bla79] G. R. Blakley. Safeguarding cryptographic keys. In *International Workshop on Managing Requirements Knowledge, MARK*, 1979.
- [DN98] Cynthia Dwork and Moni Naor. An efficient existentially unforgeable signature scheme and its applications. *J. Cryptol.*, 11(3):187–208, 1998.
- [Fel87] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *FOCS*, 1987.
- [ISN89] Mitsuru Ito, Akira Saito, and Takao Nishizeki. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan Part Iii-fundamental Electronic Science*, 1989.
- [JJK⁺17] Zahra Jafargholi, Chethan Kamath, Karen Klein, Ilan Komargodski, Krzysztof Pietrzak, and Daniel Wichs. Be adaptive, avoid overcommitting. In *CRYPTO*, 2017.
- [KNY14] Ilan Komargodski, Moni Naor, and Eylon Yogev. Secret-sharing for NP. In *ASIACRYPT*, 2014.
- [Kra93] Hugo Krawczyk. Secret sharing made short. In *CRYPTO*, 1993.
- [LOS⁺10] Allison B. Lewko, Tatsuaki Okamoto, Amit Sahai, Katsuyuki Takashima, and Brent Waters. Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption. In *EUROCRYPT*, 2010.
- [Ped91] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO*, 1991.
- [RB07] Phillip Rogaway and Mihir Bellare. Robust computational secret sharing and a unified account of classical secret-sharing goals. In *ACM CCS*, 2007.
- [Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, 1979.

- [VNS⁺03] Vinod Vaikuntanathan, Arvind Narayanan, K. Srinathan, C. Pandu Rangan, and Kwangjo Kim. On the power of computational secret sharing. In *INDOCRYPT*, 2003.
- [Yao82] Andrew C Yao. Protocols for secure computations. In *FOCS*. IEEE, 1982.
- [Yao86] Andrew Chi-Chih Yao. How to generate and exchange secrets. In *FOCS*. IEEE, 1986.
- [Yao89] Andrew Chi-Chih Yao. Unpublished manuscript, 1989.

Appendix

A Parameterized Adaptive Transformation

In this section, we describe parameterized version of our [Construction 4.1](#) transformation which can lower overhead by reducing the number of copies of the underlying semi-adaptive scheme which need to be instantiated. Specifically, we look at trading off the size of our scheme for two factors, a reduction tightness parameter c and collusion bound q .

- **Reduction Tightness:** We introduce a tightness parameter c , which controls the advantage of the underlying static scheme compared to the resulting adaptive scheme. We can view c as a sliding scale between our original construction ($c = 1$), where our adaptive scheme is as secure (up to polynomial factors) as the underlying static scheme, and $c = n$, corresponding exactly to the generic subexponential approach of achieving adaptivity through guessing the complete challenge input x .
- **Collusion Bound:** The collusion bound q represents a limit to the number of corruption queries an adversary is able to perform. Since our original scheme uses one ‘layer’ to erase the information of one corruption query, if we a-priori have a bound on the number of corruption queries the adversary can perform lower than the total number of shares n , we can reduce the number of ‘layers’ of our scheme to match.

Definition A.1 (ϵ -Secure). For any $\epsilon = \epsilon(\lambda)$, we say a scheme is ϵ secure if for all efficient adversaries $\mathcal{A}$ in security game Exp , there exists a negligible function negl such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{Exp}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\text{Exp}_{\mathcal{A}}(\lambda, 1) = 1]| \leq \epsilon \cdot \text{negl}(\lambda)$$

We can view ‘normal’ polynomial security to be of a primitive as being 1-secure.

Definition A.2 (Bounded Collusion (Adaptive) Security). We say a scheme is secure against collusion bound q if the scheme is secure for adversaries who corrupt up to q shares.

Definition A.3 (Decomposable CSS with c -bounded adaptivity). We consider a modification to the semi-adaptive security definition in [Section 3](#), where the adversary can additionally make up to c post-challenge corruption queries.

1. Algorithm $\mathcal{A}(1^\lambda)$ sends a party count 1^n to the challenger, who samples $(sh_1, \dots, sh_n), sk \leftarrow \text{ShareKey}(1^\lambda, 1^n)$
2. The adversary $\mathcal{A}$ can make these queries as much as they want:
 - **Public Share Query:** $\mathcal{A}$ requests a secret s and a program P , and the challenger computes $ps \leftarrow \text{ShareMsg}(sk, P, s)$ and responds with ps
 - **Corruption Query:** $\mathcal{A}$ requests an index $i \in [n]$, and the challenger responds with sh_i .
3. The adversary $\mathcal{A}$ requests a challenge in the form of two secrets s_0, s_1 and a programs P^* . The challenger replies with $ps^* \leftarrow \text{ShareMsg}(sk, P^*, s_b)$.
4. The adversary $\mathcal{A}$ can continue to make queries to the **Public Share Query** oracle of Step 2 as many times as desired **and up to c queries to the Corruption Queries oracle.**
5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let C be the set of indices for which $\mathcal{A}$ issued a corruption query on, and let x be the characteristic vector of C . If $P^*(x_i) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0.

Lemma A.4. Suppose Π_{SA} is a semi-adaptively ϵ -secure decomposable secret sharing scheme. Then for any $c \in [n]$, Π_{SA} is also semi-adaptively $\epsilon \cdot n^{-c}$ -secure with c post-challenge corruption queries.

Proof. This follows from a simple guessing argument - our reduction will simply guess the c adaptive corruption queries with probability $\frac{1}{n^c}$, in which case they will be able to simulate the plain semi-adaptive challenger. More formally, consider the following sequence of experiments:

- $\text{Hyb}_{c'}^{(b)}$: In this experiment, the challenger guesses the first c' corruption queries made by $\mathcal{A}$, and aborts if they guess incorrectly.
1. Algorithm $\mathcal{A}(1^\lambda)$ sends a party count 1^n to the challenger, who samples $(\text{sh}_1, \dots, \text{sh}_n), \text{sk} \leftarrow \text{ShareKey}(1^\lambda, 1^n)$
 - The challenger samples a random sequence of c' parties $(i_1, \dots, i_{c'}) \xleftarrow{\mathcal{R}} [n]^{c'}$.
 2. The adversary $\mathcal{A}$ can make these queries as much as they want:
 - **Public Share Query:** $\mathcal{A}$ requests a secret s and a program P , and the challenger computes $\text{ps} \leftarrow \text{ShareMsg}(\text{sk}, P, s)$ and responds with ps .
 - **Corruption Query:** $\mathcal{A}$ requests an index $i \in [n]$, and the challenger responds with sh_i .
 3. The adversary $\mathcal{A}$ requests a challenge in the form of two secrets s_0, s_1 and a programs P^* . The challenger replies with $\text{ps}^* \leftarrow \text{ShareMsg}(\text{sk}, P^*, s_b)$.
 4. The adversary $\mathcal{A}$ can continue to make queries to the **Public Share Query** oracle of Step 2 as many times as desired and up to c queries to the **Corruption Queries** oracle.
 - For $j \leq c'$, on the j^{th} query to the corruption query oracle, the challenger aborts with output 0 if $\mathcal{A}$ queries on a party other than i_j .
 5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0.

Claim A.5. For all adversaries $\mathcal{A}$, $\Pr[\text{Hyb}_{c'-1}^{(b)}(\mathcal{A}) = 1] \geq \frac{1}{n} \Pr[\text{Hyb}_{c'}^{(b)}(\mathcal{A}) = 1]$

Proof. Since $i_{c'}$ is uniform and independent of the queries of $\mathcal{A}$, if $\mathcal{A}$ makes $\geq c'$ corruption queries, the probability $\mathcal{A}$ does not abort on the c'^{th} post-challenge corruption query is exactly $\frac{1}{n}$. $\square$

Claim A.6. Suppose Π_{SA} is a semiadaptively ϵ -secure decomposable secret sharing scheme. Then for any efficient adversary $\mathcal{A}$, there exists negligible function $\text{negl}(\cdot)$ such that

$$\left| \Pr[\text{Hyb}_c^{(0)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_c^{(1)}(\mathcal{A}) = 1] \right| \leq \text{negl}(\lambda)$$

Proof. Let $\mathcal{A}$ be an adversary against the semi-adaptive security of Π_{SA} . Consider the following reduction $\mathcal{B}$:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per the semi-adaptive secret sharing game with c post-challenge corruption queries.
1. Adversary $\mathcal{A}$ sends a party count 1^n to $\mathcal{B}$.
 - $\mathcal{B}$ samples a random sequence of c parties $(i_1, \dots, i_c) \xleftarrow{\mathcal{R}} [n]^c$.
 - For each $j \in [c]$, $\mathcal{B}$ submits a **Corruption Query** to the semi-adaptive challenger on i_j , receiving responses $\{\text{sh}_{i_j}\}_{j \in [c]}$.
2. The adversary $\mathcal{A}$ sends $\mathcal{B}$ **Public Share Query** and **Corruption Query**, which reduction $\mathcal{B}$ directly forwards to the semi-adaptive challenger, returning the challenger's response to $\mathcal{A}$.
3. The adversary $\mathcal{A}$ requests a challenge with secrets s_0, s_1 and program P^* . As before, $\mathcal{B}$ simply sends this to the semi-adaptive challenger, forwarding their response
4. The reduction $\mathcal{B}$ continues to respond to $\mathcal{A}$'s queries as follows:
 - **Public Share Query:** as in Step 2, $\mathcal{B}$ directly forwards these queries to the semi-adaptive challenger and returns their response.
 - **Corruption Query:** On the j^{th} such post-corruption query, $\mathcal{A}$ sends an index $i \in [n]$. If $i = i_j$, then $\mathcal{B}$ returns sh_{i_j} as received in Step 1. Otherwise, $\mathcal{B}$ aborts with output 0.

5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then $\mathcal{B}$ outputs b' . Otherwise $\mathcal{B}$ outputs 0.

Observe that $\mathcal{B}$ aborts exactly as $\text{Hyb}_c^{(b)}$, and otherwise, the view of $\mathcal{A}$ is exactly the subset of shares they request in the semi-adaptive decomposable secret sharing game. $\square$

Taking together [Claims A.5](#) and [A.6](#) gives us the proof of our claim. $\square$

Construction A.7. We construct an adaptively secure CSS scheme with collusion bound q and tightness parameter c as follows. We use the same building block - a semi-adaptively secure decomposable secret sharing scheme supporting the same programs class $\mathcal{P}$ and secret space $\mathcal{S}$ - as [Construction 4.1](#). For cleaner notation, we will assume parameter c divides q , but this can be rounded with constant overhead. As in our original construction, our scheme will make use of the following ingredient:

- $\Pi_{\text{SA}} = (\text{SA.ShareKey}, \text{SA.ShareMsg}, \text{SA.Recon})$ is a semi-adaptively secure decomposable secret sharing scheme supporting programs $\mathcal{P}$ and secret space $\mathcal{S}$. We also require share size shsize of this scheme to be encodeable in the secret space $\{0, 1\}^{\text{shsize}} \subseteq \mathcal{S}$. Since the share size of a CSS scheme can be made independent of the secret space generically with symmetric-key encryption (in fact our transformation [Construction 3.1](#) does this), this does not impose any additional assumptions.

Our construction is as follows:

- $\text{Share}(1^\lambda, P, s)$: On input security parameter 1^λ , policy P (supporting n parties), and secret s , the Share algorithm does as follows:
 - Set the number of ‘layers’ of our scheme $n' = q/c$
 - For $j \in [0, n']$, compute $\{\text{sh}_i^j\}, \text{sk}^j \leftarrow \text{SA.ShareKey}(1^\lambda, 1^n)$ where sh_i^j is the i -th output of the algorithm on iteration j .
 - For $i \in [n], j \in [n']$, compute $\text{ps}_i^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, \text{sh}_i^{j-1})$.
 - Compute $T \leftarrow \text{SA.ShareMsg}(\text{sk}^0, P, s)$.
 - Output $\text{ps} = T, \text{sh}_i = (\text{sh}_i^{n'}, (\text{ps}_i^1 \dots, \text{ps}_i^{n'}))$.
- $\text{Recon}(P, X, \text{ps}, \{\text{sh}_i\}_{i \in X})$: On input policy P , authorized set X , public information ps and shares $\{\text{sh}_i\}$ for the set X , the reconstruction algorithm does as follows:
 - Parse $\text{ps} = T$ and $\text{sh}_i = (\text{sh}_i^{n'}, (\text{ps}_i^1 \dots, \text{ps}_i^{n'}))$.
 - Iterating j from n' to 1 and for all $i \in X$:
 - * Recover $\text{sh}_i^j \leftarrow \text{SA.Recon}(P, X, \text{ps}_i^j, \{\text{sh}_i^j\}_{i \in X})$.
 - Recover and output $s \leftarrow \text{SA.Recon}(P, X, T, \{\text{sh}_i^{n'}\}_{i \in X})$.

Theorem A.8 (Security). *Suppose Π_{SA} is a semi-adaptively ϵ -secure decomposable computational secret sharing scheme. Then [Construction A.7](#) is an q query adaptively $(\epsilon \cdot n^{-c+1})$ -secure CSS scheme.*

Proof. To show adaptive security, we will proceed through a sequence of hybrids.

- $\text{Hyb}_{\text{real}}^{(b)}$ - This is the real adaptive security game, where the challenger gives a secret sharing of s_b .
 1. Algorithm $\mathcal{A}(1^\lambda)$ sends a program P on n inputs and a pair of secrets s_0, s_1 to the challenger, who samples a Share program.
 - Set $n' = q/c$.
 - For $j \in [0, n']$, compute $\{\text{sh}_i^j\}, \text{sk}^j \leftarrow \text{SA.ShareKey}(1^\lambda, 1^n)$ where sh_i^j is the i -th output of the algorithm on iteration j .

- For $i \in [n], j \in [n']$, compute $\text{ps}_i^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, \text{sh}_i^{j-1})$.
 - Compute $T \leftarrow \text{SA.ShareMsg}(\text{sk}^0, s_b)$.
 - Return $\text{ps} = T$ to the $\mathcal{A}$.
2. For up to q times, the adversary $\mathcal{A}$ issues query i to the challenger. The challenger responds with $\text{sh}_i = (\text{sh}_i^{n'}, (\text{ps}_i^1 \dots \text{ps}_i^{n'}))$.
 3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of parties for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0.
- $\text{Hyb}_\ell^{(b)}$ - For $\ell \in [q]$, in this hybrid, the adversary will switch the way they sample the $(q - \ell + 1)^{\text{th}}$ corruption query. Here, they will sample the all but the first $\lfloor \ell/c \rfloor$ ‘subshares’ as normal and replace all the others with sharings of 0.
 - ...
 - 2. For up to q times, the adversary $\mathcal{A}$ issues corruption queries to the challenger. On the k^{th} corruption query (to party i_k) made on:
 - If $k < (q - \ell + 1)$, compute sample sh_{i_k} as in $\text{Hyb}_{\text{real}}^{(b)}$.
 - Otherwise for $j \in [n']$:
 - * For $j \cdot c > k$ set $\text{ps}_{i_k}^j = \text{ps}_{i_k}^j$ as before.
 - * For $j \cdot c \leq k$ sample $\text{ps}_{i_k}^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, 0^{\text{shsize}})$.
 - * Output $\text{sh}_{i_k} = (\text{sh}_{i_k}^{n'}, (\text{ps}_{i_k}^1 \dots \text{ps}_{i_k}^{n'}))$.
 - ...

Lemma A.9. For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$,

$$\Pr[\text{Hyb}_{\text{real}}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_0^{(b)}(\mathcal{A}) = 1]$$

Proof. Since $\mathcal{A}$ can only make up to q corruption queries, these hybrids are identical. $\square$

Lemma A.10. Suppose Π_{SA} is a semi-adaptively secure decomposable secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$ and $\ell \in [q]$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1] \right| \leq \frac{1}{n^c} \cdot \text{negl}(\lambda).$$

Proof. First, note that if the adversary makes less than $q - \ell + 1$ corruption queries, the adversary’s view of these hybrids is identical. Thus, we will focus on the case the adversary makes at least $q - \ell + 1$ corruption queries. To show this indistinguishability, we will proceed through a sequence of subhybrids between $\text{Hyb}_{\ell-1}^{(b)}$ and $\text{Hyb}_\ell^{(b)}$, iterating through $k^* \in [0, \lceil (q - \ell + 1)/c \rceil]$ defined as follows:

- $\text{sHyb}_{\ell-1, k^*}^{(b)}$ - This hybrid proceeds as $\text{Hyb}_{\ell-1}^{(b)}$, except on the $(q - \ell + 1)^{\text{th}}$ corruption query (corrupting party i_ℓ), the adversary will replace sharing $j \in [k^*]$ with a sharing of 0^{shsize} .
- ...
- 2. For up to q times, the adversary $\mathcal{A}$ issues corruption queries to the challenger. On the k^{th} query (on party i_k):
 - If $k \neq \ell$, compute sample sh_i as in $\text{Hyb}_{\ell-1}^{(b)}$.
 - Otherwise when $k = \ell$, for $j \in [n']$:
 - * For $j \in [k^*]$ sample $\text{ps}_{i_k}^j \leftarrow \text{SA.ShareMsg}(\text{sk}^j, P, 0^{\text{shsize}})$.
 - * Otherwise, set $\text{ps}_{i_k}^j$ as before.

* Output $\text{sh}_{i_k} = (\text{sh}_{i_k}^{n'}, (\text{ps}_{i_k}^1 \dots, \text{ps}_{i_k}^{n'}))$.
 $\dots$

Claim A.11. For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$ and $\ell \in [q]$,

$$\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{sHyb}_{\ell-1,0}^{(b)}(\mathcal{A}) = 1].$$

Proof. Since $[0] = \emptyset$, this hybrid is computed exactly as $\text{Hyb}_{\ell-1}^{(b)}$ □

Claim A.12. Suppose Π_{SA} is a semi-adaptively secure decomposable secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$, $\ell \in [q]$, and $k^* \in \lceil [(q - \ell + 1)/c] \rceil$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{sHyb}_{\ell-1,k^*-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{sHyb}_{\ell-1,k^*}^{(b)}(\mathcal{A}) = 1]| = n^{-c} \cdot \text{negl}(\lambda).$$

Proof. To prove this statement, we will reduce the distinguishability of these hybrids to the n^{-c} semi-adaptive security of Π_{SA} with c post-challenge corruptions. By [Lemma A.4](#), this follows from semi-adaptive security of Π_{SA} .

Suppose that there exists an efficient adversary $\mathcal{A}$ that can distinguish these two experiments with non-negligible probability ε . We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that breaks security of Π_{SA} with the same advantage:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per $\text{sHyb}_{\ell-1,\lceil \ell/c \rceil - 1}^{(b)}$.
1. Algorithm $\mathcal{A}(1^\lambda)$ sends a program P and a pair of secrets s_0, s_1 to $\mathcal{B}$, who computes the following:
 - For $j \in [0, n'] \setminus \{k^*\}$, compute $\{\text{sh}_i^j\}, \text{sk}_i^j \leftarrow \text{SA.ShareKey}(1^\lambda, 1^n)$ where sh_i^j is the i -th output of the algorithm on iteration j .
 - For $i \in [n], j \in [n'] \setminus \{k^*, k^* + 1\}$, compute $\text{ps}_i^j \leftarrow \text{SA.ShareMsg}(\text{sk}_i^j, P, \text{sh}_i^{j-1})$.
 - Compute $T \leftarrow \text{SA.ShareMsg}(\text{sk}^{n'}, P, s_b)$.
 - Return $\text{ps} = T$ to the $\mathcal{A}$.
2. For up to q times, the adversary $\mathcal{A}$ makes corruption queries to $\mathcal{B}$. On corruption query k , $\mathcal{B}$ computes $(\text{sh}_{i_k}^1, \dots, \text{sh}_{i_k}^{n'})$ as follows:
 - (a) For $j \neq \{k^*, k^* + 1\}$, the reduction $\mathcal{B}$ sets $\text{ps}_{i_k}^j$ as $\text{sHyb}_{\ell-1,k^*-1}^{(b)}$.
 - (b) To compute $\text{ps}_{i_k}^{k^*+1}$, $\mathcal{B}$ uses the index k and computes the following:
 - i. If $k \cdot c \leq q - \ell + 1$, $\mathcal{B}$ submits a corruption query to the Semi-Adaptive CSS challenger on party i_k , receiving response $\text{sh}_{i_k}^{k^*}$. $\mathcal{B}$ uses this to set $\text{ps}_{i_k}^{k^*+1} \leftarrow \text{SA.ShareMsg}(\text{sk}_{i_k}^{k^*+1}, P, \text{sh}_{i_k}^{k^*})$.
 - ii. For $k \cdot c > q - \ell + 1$, $\mathcal{B}$ instead computes $\text{ps}_{i_k}^{k^*+1} \leftarrow \text{SA.ShareMsg}(\text{sk}_{i_k}^{k^*+1}, P, 0^{\text{shsize}})$.
 - (c) To compute $\text{ps}_{i_k}^{k^*}$, $\mathcal{B}$ submits a secret or challenge query to the Semi-Adaptive CSS challenger, and sets the response as $\text{ps}_{i_k}^{k^*}$ ⁵.
 - i. If $k \cdot c < q - \ell + 1$, submit public share query on policy P and secret $\text{sh}_i^{k^*-1}$.
 - ii. If $k \cdot c = q - \ell + 1$, $\mathcal{B}$ submits challenge query on policy P and secrets $s_0 = \text{sh}_i^{k^*-1}, s_1 = 0^{\text{shsize}}$.
 - iii. If $k \cdot c > q - \ell + 1$, submit public share query on policy P and secret 0^{shsize} .
3. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of parties for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then the output of the experiment is b' . Otherwise the experiment outputs 0. $\mathcal{B}$ returns the output of the experiment.

Our reduction here relies on the observations that:

⁵Skip this step if $k^* = n$

- $\text{sHyb}_{\ell-1, k^*-1}^{(b)}$ and $\text{sHyb}_{\ell-1, k^*}^{(b)}$ differ only on the value of $\text{ps}_i^{k^*}$, as encodings of $\text{sh}_i^{k^*}$ or 0^{shsize} respectively.
- On corruption queries k after the $(q - \ell + 1)^{\text{th}}$, there are at most c such queries where k is also $> c \cdot \lfloor \ell/c \rfloor$. Since $\text{ps}_{i_k}^{k^*+1}$ have already been switched to sharings of 0^{shsize} for $k^* \geq k > c \cdot \lfloor \ell/c \rfloor$, the reduction never has to give out such share keys $\text{sh}_{i_k}^{k^*}$. Thus, after $\mathcal{B}$ issues a challenge query in step (c).ii, it has makes at most c more corruption queries in step (b).i. Thus, $\mathcal{B}$ is a valid challenger in the semi-adaptive security game which makes $\leq c$ post-challenge corruption queries.

Taking these two together, we can see that $\mathcal{B}$ is a valid semi-adaptive adversary which simulates $\text{sHyb}_{\ell-1, k^*-1}^{(b)}$ when $b = 0$ and $\text{sHyb}_{\ell-1, k^*}^{(b)}$ when $b = 1$. $\square$

Claim A.13. For all adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$ and $\ell \in [q]$,

$$\Pr[\text{sHyb}_{\ell-1, q-\ell+1}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_{\ell}^{(b)}(\mathcal{A}) = 1].$$

Proof. Observe here, on Corruption Query $q - \ell + 1$, $\text{ps}_{i_\ell}^j$ is set to $\text{SA.ShareMsg}(\text{sk}^j, P, 0^{\text{shsize}})$ for $j \cdot c \in d[\ell, q]$. Since j is always $\leq n'$, this is equivalent to the check that $j \cdot c \geq k$ in $\text{Hyb}_{\ell}^{(b)}$. $\square$

Taking together [Claims A.11](#) to [A.13](#), we get the proof of our [Lemma 4.6](#). $\square$

Lemma A.14. Suppose Π_{SA} is a semi-adaptively secure decomposable secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$,

$$|\Pr[\text{Hyb}_q^{(0)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_q^{(1)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

This proceeds analogously to the proof of [Lemma 4.10](#).

Taking together [Lemmas A.9](#), [A.10](#) and [A.14](#), we get the [Theorem A.8](#). $\square$

B Decomposable Secret Sharing Proof of Security

In this section, we show the hybrid indistinguishability arguments in the proof of security of [Construction 3.1](#) deferred from the main body.

Claim B.1 ([Claim 3.4](#) restated). For all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$,

$$\Pr[\text{Hyb}_{\text{real}}^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_0^{(b)}(\mathcal{A}) = 1].$$

Proof. Here, note that since $i > 0$, the new condition is never true, and so these experiments are identical. $\square$

Claim B.2 ([Claim 3.5](#) restated). Suppose Π_{SKE} is an IND-CPA secure symmetric key encryption scheme. Then for all efficient adversaries $\mathcal{A}$, bits $b \in \{0, 1\}$, and $\ell \in [n]$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_{\ell}^{(b)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

Proof. Suppose that there exists an efficient adversary $\mathcal{A}$ that can distinguish these two experiments with non-negligible probability ε . Here, we break our analysis into two cases conditioning on the event E_{ℓ} , if $\mathcal{A}$ submits a corruption query on index ℓ . First, observe that E_{ℓ} occurs - i.e. if $\mathcal{A}$ does submit a corruption query on index ℓ - these games are identical, as the adversary's response only changes when $\mathcal{A}$ does not request a corruption query. In the case E_{ℓ} does not occur, we use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that breaks security of Π_{SKE} with the same advantage:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per $\text{Hyb}_{\ell-1}^{(b)}$.
1. Adversary $\mathcal{A}$ sends a party count 1^n to $\mathcal{B}$. For all $i \in [n] \setminus \{\ell\}$, $\mathcal{B}$ generates $\text{sk}_i \leftarrow \text{SKE.KeyGen}(1^\lambda)$.
2. The adversary $\mathcal{A}$ can make these queries as much as they want:

- **Public Share Query:** $\mathcal{A}$ requests a secret s and a program P , which $\mathcal{B}$ responds to by
 - Compute sharing $(S.ps, S.sh_1, \dots, S.sh_n) \leftarrow S.Share(1^\lambda, P, s)$.
 - For $i \in [n] \setminus \{\ell\}$, encrypt $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, S.sh_i)$.
 - $\mathcal{B}$ submits an encryption query to the IND-CPA challenger on message $S.sh_\ell$, and receives a ciphertext which it sets as ct_ℓ .
 - Return $ps = (S.ps, \{ct_i\}_{i \in [n]})$.
- **Corruption Query:** $\mathcal{A}$ requests an index $i \in [n]$, and the challenger responds with sk_i (recall from our event $\mathcal{A}$ never requests sk_ℓ).

3. The adversary $\mathcal{A}$ requests a challenge with secrets s_0, s_1 and program P^* .

- Compute sharing $(S.ps, S.sh_1, \dots, S.sh_n) \leftarrow S.Share(1^\lambda, P, s)$.
- For $i \in [n]$:
 - If $i < \ell$ and $\mathcal{A}$ has not requested a **Corruption Query** on i , encrypt $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, 0^{\text{shsize}})$.
 - $\mathcal{B}$ submits a challenge query to the IND-CPA challenger with $m_0 = S.sh_\ell, m_1 = 0$. They set the ciphertext they receive as ct_ℓ .
 - For all other i , $\mathcal{B}$ sets $ct_i \leftarrow \text{SKE.Encrypt}(sk_i, S.sh_i)$.
- Return $ps = (S.ps, \{ct_i\}_{i \in [n]})$.

4. The reduction $\mathcal{B}$ responds to $\mathcal{A}$'s **Secret Queries** oracle queries exactly as they did in Step 2.

5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. Let X be the set of indices for which $\mathcal{A}$ issued a corruption query on. If $P^*(X) = 0$, then $\mathcal{B}$ outputs b' . Otherwise $\mathcal{B}$ outputs 0.

Observe that in $\text{Hyb}_{\ell-1}^{(b)}$, ct_ℓ is always an encryption $S.sh_\ell$ under sk_ℓ , while in $\text{Hyb}_\ell^{(b)}$, ct_ℓ is an encryption of 0 if it is uncorrupted - i.e. E_ℓ does not occur. These correspond exactly to the message encrypted under ct_ℓ by the reduction. From our two cases, we can see that

$$\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | E_\ell] = \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | E_\ell]$$

and

$$|\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | \neg E_\ell] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | \neg E_\ell]| = \text{negl}(\lambda)$$

Combining these, we can use the fact that

$$\Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1] = \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | E_\ell] \cdot \Pr[E_\ell] + \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | \neg E_\ell] \cdot \Pr[\neg E_\ell]$$

(and equivalently for $\text{Hyb}_{\ell-1}^{(b)}$) to simplify

$$\begin{aligned} & |\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1]| = \\ & |\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | E_\ell] \cdot \Pr[E_\ell] + \Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | \neg E_\ell] \cdot \Pr[\neg E_\ell] \\ & - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | E_\ell] \cdot \Pr[E_\ell] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | \neg E_\ell] \cdot \Pr[\neg E_\ell]| = \\ & |\Pr[E_\ell] \cdot (\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | E_\ell] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | E_\ell]) \\ & + \Pr[\neg E_\ell] \cdot (\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | \neg E_\ell] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | \neg E_\ell])| = \\ & \Pr[\neg E_\ell] \cdot |\Pr[\text{Hyb}_{\ell-1}^{(b)}(\mathcal{A}) = 1 | \neg E_\ell] - \Pr[\text{Hyb}_\ell^{(b)}(\mathcal{A}) = 1 | \neg E_\ell]| = \\ & \Pr[\neg E_\ell] \cdot \text{negl}(\lambda) = \\ & \text{negl}(\lambda). \end{aligned}$$

□

Claim B.3 (Claim 3.6 restated). Suppose Π_S is a statically secure computational secret sharing scheme. Then for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$|\Pr[\text{Hyb}_n^{(0)}(\mathcal{A}) = 1] - \Pr[\text{Hyb}_n^{(1)}(\mathcal{A}) = 1]| = \text{negl}(\lambda).$$

Proof. Suppose that there exists an efficient adversary $\mathcal{A}$ that can distinguish these two experiments with non-negligible probability ϵ . We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that breaks security of Π_S with the same advantage:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per $\text{Hyb}_n^{(b)}$.
1. Adversary $\mathcal{A}$ sends a party count 1^n to $\mathcal{B}$. For all $i \in [n]$, $\mathcal{B}$ generates $\text{sk}_i \leftarrow \text{SKE.KeyGen}(1^\lambda)$.
2. The adversary $\mathcal{A}$ can make these queries as much as they want:
 - **Public Share Query:** $\mathcal{A}$ requests a secret s and a program P , which the $\mathcal{B}$ responds to by
 - Compute sharing $(\text{S.ps}, \text{S.sh}_1, \dots, \text{S.sh}_n) \leftarrow \text{S.Share}(1^\lambda, P, s)$.
 - For $i \in [n]$, encrypt $\text{ct}_i \leftarrow \text{SKE.Encrypt}(\text{sk}_i, \text{S.sh}_i)$.
 - Return $\text{ps} = (\text{S.ps}, \{\text{ct}_i\}_{i \in [n]})$.
 - **Corruption Query:** $\mathcal{A}$ requests an index $i \in [n]$, and $\mathcal{B}$ responds with sk_i .
3. The adversary $\mathcal{A}$ requests a challenge with secrets s_0, s_1 and program P^* .
 - Let X be the set of indices for which $\mathcal{A}$ issued a corruption query on.
 - $\mathcal{B}$ submits secrets s_0, s_1 , program P^* and set X to the static CSS security challenger, and receives $\text{S.ps}, \{\text{S.sh}_i\}_{i \in X}$.
 - For $i \in [n]$:
 - If $i \in X$, $\mathcal{B}$ sets $\text{ct}_i \leftarrow \text{SKE.Encrypt}(\text{sk}_i, \text{S.sh}_i)$.
 - Otherwise, $\mathcal{B}$ sets $\text{ct}_i \leftarrow \text{SKE.Encrypt}(\text{sk}_i, 0^{\text{shsize}})$.
 - $\mathcal{B}$ returns $\text{ps} = (\text{S.ps}, \{\text{ct}_i\}_{i \in [n]})$ to the adversary.
4. The reduction $\mathcal{B}$ responds to $\mathcal{A}$'s **Secret Queries** oracle queries exactly as they did in Step 2.
5. Adversary $\mathcal{A}$ outputs a bit $b' \in \{0, 1\}$. If $P^*(X) = 0$, then $\mathcal{B}$ outputs b' . Otherwise $\mathcal{B}$ outputs 0.

First, note that in $\text{Hyb}_n^{(b)}$, in the challenge, it is always the case that $i \leq n$, thus, whether ct_i is an encryption of 0 or S.sh_i is only dependent on if $\mathcal{A}$ issued a **Corruption Query** on i . Thus, we can see that these hybrids only differ in what secret is being shared in the CSS scheme, which $\mathcal{B}$ simulates exactly. Since X is exactly the set of corruption queries, we can see that when $P^*(X) = 0$ corresponds exactly to when $\mathcal{B}$ is a valid static CSS adversary, so the advantage of $\mathcal{B}$ is equal to the advantage of $\mathcal{A}$. □